

VOGUE: VirtIO-GPU on Unikraft for Graphics Virtualization

Submission: #001

Abstract

Unikernels promise small, fast virtual machines, but graphics and accelerator workloads usually depend on Linux DRM/KMS, Mesa, and large driver stacks. VOGUE asks how far a Unikraft guest can go with a bounded VirtIO-GPU frontend instead of importing those stacks. It contributes a compact VirtIO-GPU library, DRM/GBM/EGL compatibility shims for application ports, a Venus command/ring substrate, and a static Vulkan dispatch layer for llama.cpp/ggml’s ggml-vulkan backend.

The artifact is evidence-gated. On a host with the Venus stack — a Venus-capable QEMU (`virtio-gpu-gl-pci,venus=true`), a Venus-enabled virglrenderer, and an accessible GPU render node — its generated 27-row matrix is 27/27 PASS; without that stack the GPU/transport rows fall back to structured blocked.* status rather than faking a result. Host-native tests validate protocol ABI, 2D paths, Venus encoding/ring behavior, Vulkan ICD bootstrap, and llama/ggml substrates. Application rows cover bounded kmscube, glmark2, vkmark, and llama.cpp case studies. For ggml-vulkan, VOGUE covers the required upstream Vulkan API surface, passes a 164-check static-dispatch regression, and — once the historical modern-PCI transport blocker is cleared — runs the upstream llama.cpp Vulkan bench appliance end-to-end on a real GPU over virtio-gpu-gl Venus (pp512≈245 t/s, tg128≈3.1 t/s on a Tesla V100), with the Vulkan server appliance loading all model layers onto the GPU and reaching model-loaded readiness. We report implementation coverage and substrate correctness separately from GPU-throughput claims, and every throughput number is backed by a same-run guest artifact.

CCS Concepts: • Software and its engineering → Virtual machines; Virtual memory; • Computer systems organization → Heterogeneous (hybrid) systems.

Keywords: unikernels, VirtIO-GPU, graphics virtualization, library operating systems

ACM Reference Format: Paper #001 2026 VOGUE: VirtIO-GPU on Unikraft for Graphics Virtualization . In *ACM Symposium on Operating Systems Principles (SOSP)*, September 30-October 3, 2026, TBD. ACM, New York, NY, USA, 34 pages. <https://doi.org/10.1145/0000000000>



This work is licensed under a [Creative Commons Attribution International 4.0 License](https://creativecommons.org/licenses/by/4.0/).

SOSP, September 30-October 3, 2026, TBD.

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-0000-0000-0/00/00...\$15.00

<https://doi.org/10.1145/0000000000>

1 Introduction

Unikernels — library operating systems compiled together with a single application into a minimal VM image — have demonstrated compelling advantages for server workloads: images under 10 MB, memory footprints of a few tens of megabytes, and boot times of 3-40 ms [12]. These properties make them attractive for cloud microservices, edge functions, and security-sensitive appliances where specialization reduces both attack surface and resource cost.

Yet an entire class of modern workloads remains out of reach: applications that require accelerated graphics and display output. Interactive edge VMs, cloud gaming streaming, browser isolation containers, graphical test appliances, and ML-adjacent visualization pipelines all need a display substrate and, ideally, GPU-backed rendering. Today, running these workloads on a unikernel requires either a full Linux guest — forfeiting the footprint and boot-time benefits — or accepting that no graphics is available at all.

1.1 The Graphics Stack Problem

The fundamental challenge is that the Linux graphics stack is deeply monolithic. A typical application path involves: the DRM/KMS kernel subsystem, GBM buffer management, EGL platform initialization, Mesa’s OpenGL/Vulkan driver stack, virglrenderer or Venus on the host, and a chain of device nodes, ioctls, and file descriptor hand-offs. Each layer assumes the existence of the others. Porting an EGL/GLES2 or Vulkan application to a unikernel naively requires importing gigabytes of Linux subsystem code.

Unikraft’s micro-library architecture makes this untenable: every library must declare its dependencies explicitly via its configuration and build files, must not assume POSIX file descriptors for GPU devices, and must interoperate with a single-address-space, cooperative model. Importing Linux DRM wholesale would destroy the very properties that motivate unikernels.

1.2 Thesis

We argue that a unikernel can support practical graphics-capable workloads *without* importing a full Linux graphics subsystem, by implementing three targeted layers and one evidence gate:

1. A *standards-based VirtIO-GPU frontend* that speaks the OASIS VirtIO-GPU protocol [20] directly through Unikraft’s existing PCI, virtio, allocator, and synchronization libraries — no Linux DRM driver needed.
2. A *minimal EGL/GLES2/GBM/DRM shim* that provides the API surface expected by application code without implementing the full Linux graphics stack.

3. A *software rasterizer* that produces correct pixel output using a CPU path, enabling display pipeline validation independent of virglrenderer availability.
4. A *stage-gated 3D/Venus interface* that implements and tests the real control-queue ABI separately from the host execution path, so each layer can be validated independently before the end-to-end run.

This thesis leads to a working display pipeline and a falsifiable, now-realised upgrade path. Each stage is evidence-gated: the Unikraft-side protocol surface and evaluation gates, the native Venus ring-buffer protocol, and — on a host with a Venus-capable QEMU and an accessible GPU — the real virtio-gpu-gl Venus transport (xport.qemu-vgpu), the K1 virgl SUBMIT_3D path with a same-run pixel-frame proof, and end-to-end upstream llama.cpp Vulkan inference on the GPU. On a host lacking that stack those rows report a structured blocked:* status instead of an unverified claim, so “Vulkan acceleration works” is asserted only where a same-run guest artifact proves it.

1.3 Contributions

This paper makes the following contributions:

1. **VOGUE**, a complete VirtIO-GPU 2D command path for Unikraft, including resource management, DMA backing, scanout, fence synchronization, and display configuration. The implementation covers all commands required to present frames: two-dimensional resource creation, backing attachment, host transfer, scanout binding, resource flush, and capability set query.
2. A **real VirtIO-GPU 3D/Venus control surface** in libukvirtio_gpu, including context create/destroy, context attach/detach, 3D submit, blob create/destroy, resource UUID, map/unmap, and ABI checks against the project protocol header. This is transport-level code, not a Mesa/Venus guest driver, and it is evaluated as readiness evidence rather than as an acceleration claim.
3. A **minimal EGL/GLES2/GBM/DRM shim** (libukegl) that routes EGL surface creation and EGL buffer swapping through the VirtIO-GPU 2D path, enables real application code to compile against standard EGL/GLES2 headers, and preserves explicit safe-stub boundaries for unsupported Linux DRM behavior.
4. A **CPU software rasterizer** (libukswrender) that produces correct BGRA pixel output for rotating-cube and flat-fill workloads, enabling display pipeline testing without a GPU.
5. **Application ports**: upstream *kmscube* source compiled against the shim (gfx.kmscube.sw evidence row) and a minimal *glmark2-es2* scene-clear port (gfx.glmark2.sw).
6. **Single-purpose llama.cpp appliances**: llm.bench.cpu, llm.server.cpu, and llm.bench.vk boot upstream llama.cpp directly into one entrypoint with no shell. Each Kraftfile compiles only the source file matching its Kconfig mode (bench.cpp vs server.cpp), and -Os -ffunction-sections -fdata-sections -Wl,--gc-sections lets the linker drop every unreached symbol. This honours Unikraft’s **one image, one purpose** principle while staying upstream-unmodified.
7. A **generator-backed Venus encoder**: libukvenus consumes Mesa’s venus-protocol generator (vk.xml + Mako templates) through scripts/gen_libukvenus.py rather than vendoring 3,000+ lines of guest encoder. Bumping the Venus protocol is one upstream git pull + make gen-libukvenus.
8. An **evidence-gated evaluation framework** that distinguishes software-render evidence (gfx.kmscube.sw), real-driver ABI/readiness evidence (proto.real-driver/vk.readiness), QEMU transport evidence (xport.qemu-vgpu), native Venus protocol evidence (proto.venus-enc/proto.venus-ring), and blocked accelerated-frame evidence (K1), preventing overclaiming while still documenting real progress.

The rest of the paper describes the design (§4, §5), implementation (§7), and evaluation (§8), followed by discussion (§9), related work (§10), and conclusion (§11).

2 Background and Motivation

2.1 Unikraft

Unikraft [12] is a micro-library OS framework that enables construction of minimal, specialized unikernel VMs. Each library — networking stack, memory allocator, filesystem — is an independent module declared via its own configuration and build files. An application selects only the libraries it needs; the result is a single ELF binary running in a VM with no kernel/user separation.

Unikraft applications measured in the EuroSys 2021 evaluation boot in 3-40 ms and consume as little as 1 MB of image space and 10 MB of RAM [12]. The architecture enforces minimality through build-system accounting: every dependency must be explicit, dead code is eliminated at link time, and the runtime has no facility for dynamic library loading.

This architecture has proven highly successful for server workloads (network services, key-value stores, serverless functions) but creates tension with graphics workloads that assume rich OS services: device nodes, file descriptors, ioctls, dynamic library search paths, and large dependency chains.

2.2 VirtIO-GPU and VirtIO-GPU-GL

VirtIO [20] is the standard para-virtualization device framework for guest-host I/O. VirtIO-GPU defines a virtual GPU device that supports two operating modes:

- **2D mode**: the guest manages a framebuffer in guest memory, transfers it to a host-managed resource, and presents it to the virtual display.
- **3D/VirGL mode**: enabled by the VIRTIO_GPU_F_VIRGL feature bit; the guest submits GPU command streams that QEMU forwards to virglrenderer, which translates them to host OpenGL calls [30].

The VirtIO-GPU protocol is defined in the OASIS VirtIO 1.3 specification [20]. Key commands include:

- `RESOURCE_CREATE_2D`: allocate a host-managed 2D resource.
- `RESOURCE_ATTACH_BACKING`: attach guest physical memory pages to a resource.
- `TRANSFER_TO_HOST_2D`: push guest framebuffer data into the host resource.
- `SET_SCANOUT`: bind a resource to a display head.
- `RESOURCE_FLUSH`: present the resource to the virtual display.
- `GET_CAPSET_INFO` / `GET_CAPSET`: query virgl capability sets.
- `CTX_CREATE` / `SUBMIT_3D`: create a virgl context and submit command buffers.

QEMU exposes VirtIO-GPU via the two-dimensional software device and the GL-capable variant with virgl acceleration [25]. The GL variant requires a host OpenGL or EGL driver; without it, QEMU falls back to a software renderer.

2.3 Target Applications

kmscube [4] is a minimal DRM/GBM/EGL/GLES2 demo that renders a rotating cube to a KMS scanout without a display server. It was specifically designed as a bare-metal display test for embedded Linux systems, making it an ideal first target for a unikernel graphics stack.

glmark2-es2 [8] is an OpenGL ES 2.0 benchmark suite with multiple scenes covering geometry, texturing, shading, and fill rate. Its scene-clear benchmark mode provides a simple, reproducible throughput measurement with no asset loading.

2.4 Why Not Linux DRM?

One might ask: why not simply port the Linux DRM/KMS stack to Unikraft? The answer has three parts:

Size. The Linux DRM subsystem spans over 800,000 lines of C across more than 2,000 files. Even the minimal GBM + EGL + Mesa subset required to run *kmscube* is on the order of 2 million lines of code with complex build systems, LLVM dependencies, and runtime shader compilation infrastructure.

Architecture mismatch. Linux DRM assumes kernel-mode driver components, GPU device files, file descriptor passing, and ioctl-based control interfaces. None of these exist in a standard Unikraft build.

Minimality violation. Importing DRM/Mesa would destroy the image size and boot-time properties that motivate unikernels in the first place. The goal of VOGUE is to show that the full graphics stack can be substituted by a narrow, application-specific shim.

3 Design Goals and Challenges

VOGUE is built around one observation: for many cloud graphics appliances the hard requirement is not a complete desktop graphics stack, but a trustworthy path from application pixels to a virtual display with an upgrade path

to virgl and Venus. The design therefore optimizes for a narrow data plane, explicit compatibility contracts, and evidence labels that prevent accidental promotion of software-render or static-readiness results into GPU claims.

3.1 Goals

We have five design goals to guide our implementation and evaluation:

DG1 — Standards-based device interface. Use the OASIS VirtIO-GPU protocol. Avoid bespoke host protocols; the virtio-gpu wire format is stable and supported by QEMU, crosvm, and cloud-hypervisor.

DG2 — Small trusted graphics substrate. Keep the guest-side graphics substrate small enough to audit. VOGUE replaces Linux DRM/KMS, GBM, EGL platform glue, Mesa, and display-server assumptions with a few micro-libraries and explicit stubs.

DG3 — Application-source compatibility at the boundary. Preserve the source-level structure of target applications where it matters: *kmscube* should still look like a DRM/GBM/EGL/GLES2 program, and the porting layer should absorb Unikraft-specific adaptation.

DG4 — Progressive validation. Validate the stack bottom-up: native command sequencing, software rasterization, shim-level application compatibility, real control-queue ABI coverage, QEMU device discovery, then virgl/Venus rendering.

DG5 — Evidence-gated claims. Separate software-render evidence (*gfx.kmscube.sw*), real-driver protocol-readiness evidence (*proto.real-driver/vk.readiness*), QEMU transport blockers (*xport.qemu-vgpu*), and accelerated rendering evidence (K1). A successful capset or ABI probe is useful, but it is not a rendering result; a software frame is useful, but it is not acceleration.

3.2 Non-Goals

- Full Linux DRM/KMS ioctl compatibility.
- A full Mesa or shader-compiler port.
- A complete Mesa/Venus Vulkan guest-driver port, Wayland/X11, multi-window compositing, or multi-GPU scheduling. VOGUE documents and tests the VirtIO-GPU blob/controlq substrate needed by Venus, but does not claim Vulkan application execution.
- Vendor-native GPU drivers, SR-IOV passthrough, or confidential GPU isolation.
- Claiming graphics acceleration before a real virgl/Venus command stream renders frames with verified virtio-gpu acceleration.

3.3 Design Invariants

Table 1 states the invariants we use to keep a small shim honest. The important point is that every compatibility shortcut must have a bounded failure mode: a stub may return a safe sentinel, but it must not silently imply a stronger capability.

3.4 Challenges

We met six challenges in building VOGUE:

Table 1: Design invariants used to make a small compatibility layer reviewable.

Invariant	Enforced by	Why it matters
<i>I1: command order</i>	native fake backend rejects invalid lifecycle	Prevents black-screen bugs from passing unit tests.
<i>I2: fenced presentation</i>	TRANSFER and FLUSH carry completion fences	Avoids reading success from asynchronous submission alone.
<i>I3: explicit stubs</i>	DRM/GBM functions document sentinel behavior	Prevents compatibility from becoming an implicit Linux port claim.
<i>I4: row-labeled claims</i>	evaluator emits gfx.kmscube.sw, proto.real-driver, xport.qemu-vgpu, vk.readiness, and K1 separately	Prevents software frames or ABI checks from being reported as GPU rendering.
<i>I5: Unikraft reuse</i>	library dependencies stay on existing PCI, virtio, allocator, mmap/POSIX support	Prevents the project from reimplementing existing Unikraft modules.

C1 — API dependency chain. Applications like *km-scube* call into the GBM, EGL, and GLES2 API surfaces — device creation, surface setup, window surface construction, buffer swapping, and numerous GL entry points. Each call must either perform real VOGUE work or fail safely without relying on Linux device files.

C2 — VirtIO-GPU command sequencing. The 2D protocol requires a strict sequence: resource creation → backing attachment → scanout binding → host transfer → resource flush. Reordering produces no visible frame or a device error.

C3 — Fence synchronization. In a cooperative Unikraft environment, fence polling must be bounded: long enough to observe device completion after each transfer and flush, but not an unbounded spin loop that can hide a broken queue.

C4 — DMA memory model. VirtIO-GPU resources are host-private. The guest must copy software-rendered pixels into DMA-backed memory and explicitly transfer them to the host. This copy is acceptable for a correctness substrate but becomes the central bottleneck for interactive workloads.

C5 — Minimality versus compatibility. Too narrow a shim fails to run real programs; too broad a shim recreates Mesa and DRM. VOGUE chooses the smallest API subset that supports concrete application rows, then names unsupported behavior as future work.

C6 — Evaluation integrity. A high-level systems paper must make negative space visible: missing baselines, missing virgl/Venus command streams, QEMU transport blockers, and missing full-benchmark runs must appear as blocked rows rather than disappearing from the evaluation.

4 System Overview

Figure 1 shows the VOGUE library stack. The system is organized as Unikraft micro-libraries, each with an explicit dependency declaration and a build file that compiles only

what is needed. The design rule is conservative: VOGUE reuses Unikraft’s existing PCI, virtio, allocator, mmap/POSIX, and synchronization infrastructure and only adds the graphics-specific protocol and compatibility layers.

4.1 Layer Responsibilities

Since the design optimizes for a narrow data plane and explicit compatibility contracts, each layer has a well-defined responsibility:

libukdma provides DMA buffer allocation and scatter-gather descriptor construction. It is the only VOGUE layer that interacts directly with guest-physical memory layout for virtio backing; it relies on Unikraft allocation primitives instead of introducing an independent memory manager.

libukvirtio_gpu implements the VirtIO-GPU protocol on top of Unikraft’s existing virtio/PCI stack: command encoding and submission, response parsing, fence creation and polling, 2D resource management, capset discovery, context management, 3D submit, resource blobs, UUID assignment, and map/unmap helpers. The 2D path is exercised by native display tests. The 3D/blob path is implemented as a real control-queue surface and checked by ABI/static/readiness gates. On the evaluation host the current generated matrix records QEMU/Venus execution, the Mesa-compatible Venus ring-buffer protocol, KMSCube virgl frame proof, and upstream llama.cpp Vulkan runtime as PASS with same-run artifacts.

libukswrender is a CPU software rasterizer that renders rotating-cube geometry into a BGRA framebuffer. It takes a rotation angle and an output pixel buffer; internally it clips triangles, performs perspective-correct interpolation, and writes packed 32-bit pixels. It has no dependency on any graphics API.

libukegl provides the EGL/GLES2/GBM/DRM shim. On EGL initialization, it opens a connection to libukvirtio_gpu and queries capsets. On EGL window surface creation, it allocates a software framebuffer and a DMA buffer. On EGL buffer swap, it invokes libukswrender to produce the current

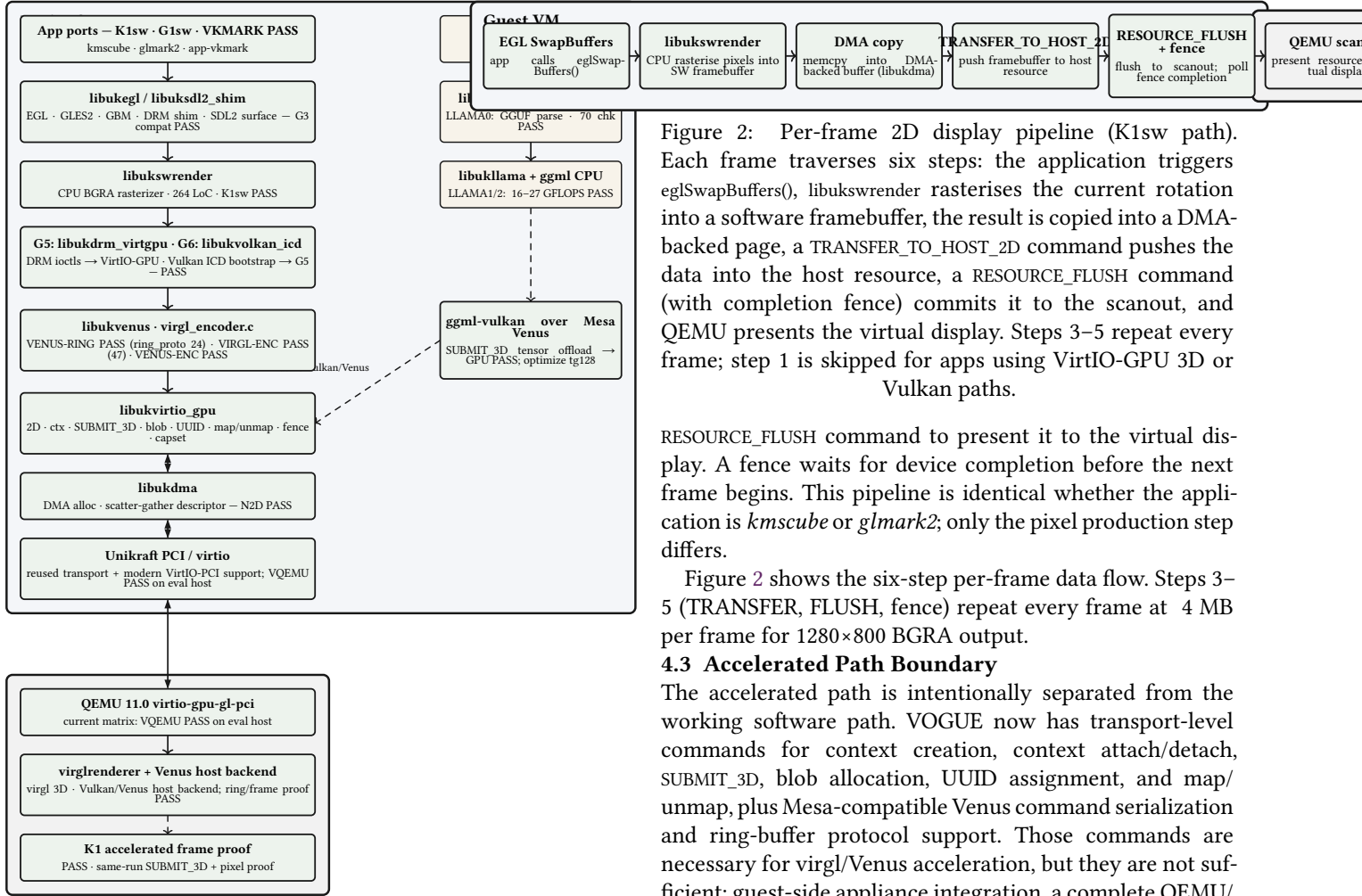


Figure 1: VOGUE software stack (current state). **Green-tinted** nodes are fully validated (solid border). **Near-white dashed** nodes are future scope. Left column: 2D display path (fully PASS: N2D→K1sw→G1sw) and the Vulkan/Venus acceleration substrate (G5+G6+libukvenus+virgl_encoder all PASS; K1 frame proof follows the current evidence matrix). Right column: LLAMA compute side-chain, including real QEMU/Venus GPU inference on the evaluation host. Both paths share libukvirtio_gpu and the Unikraft PCI/virtio transport.

frame’s pixels, then copies them to DMA memory and issues a host-transfer followed by a resource-flush command. Unsupported Linux DRM behavior remains an explicit safe stub rather than a hidden partial reimplementation.

4.2 Display Pipeline

A single software-rendered frame traverses the following path. The application calls the EGL buffer-swap routine, which triggers libukswrender to produce the current frame’s pixels at the current rotation angle. The shim copies the resulting pixel data to DMA-backed memory, then issues a TRANSFER_TO_HOST_2D command to push the framebuffer into the host-managed resource, followed by a

Figure 2: Per-frame 2D display pipeline (K1sw path). Each frame traverses six steps: the application triggers eglSwapBuffers(), libukswrender rasterises the current rotation into a software framebuffer, the result is copied into a DMA-backed page, a TRANSFER_TO_HOST_2D command pushes the data into the host resource, a RESOURCE_FLUSH command (with completion fence) commits it to the scanout, and QEMU presents the virtual display. Steps 3–5 repeat every frame; step 1 is skipped for apps using VirtIO-GPU 3D or Vulkan paths.

RESOURCE_FLUSH command to present it to the virtual display. A fence waits for device completion before the next frame begins. This pipeline is identical whether the application is *kmscube* or *glmark2*; only the pixel production step differs.

Figure 2 shows the six-step per-frame data flow. Steps 3–5 (TRANSFER, FLUSH, fence) repeat every frame at 4 MB per frame for 1280×800 BGRA output.

4.3 Accelerated Path Boundary

The accelerated path is intentionally separated from the working software path. VOGUE now has transport-level commands for context creation, context attach/detach, SUBMIT_3D, blob allocation, UUID assignment, and map/unmap, plus Mesa-compatible Venus command serialization and ring-buffer protocol support. Those commands are necessary for virgl/Venus acceleration, but they are not sufficient: guest-side appliance integration, a complete QEMU/Venus probe, and frame-proof work are still required. The paper therefore treats the accelerated path as a readiness-and-blocker result until xport.qemu-vgpu and the K1 rows pass with same-run evidence.

5 VirtIO-GPU Frontend Design

The VirtIO-GPU frontend (libukvirtio_gpu) is the lowest-level VOGUE component. It speaks directly to the virtual device with no Linux DRM driver, no Mesa userspace stack, and no private host protocol. Its design follows two rules: reuse Unikraft transport infrastructure where it already exists, and keep each stronger graphics claim behind an explicit evidence row.

5.1 Device Discovery and Initialization

On Unikraft, PCI device discovery uses the existing Unikraft PCI and virtio bus libraries. Legacy and transitional VirtIO devices can be reached through the current libvirtio_pci path; QEMU’s modern VirtIO-GPU Venus devices present modern PCI IDs such as 0x1050. This artifact carries a Unikraft libvirtio_pci patch for modern VirtIO-PCI discovery, and the current evaluation-host matrix records xport.qemu-vgpu as

Table 2: VirtIO-GPU 2D resource lifecycle. Each step maps to one protocol command; steps 4–6 repeat every frame.

Step	Guest operation	Device response
1	Allocate host 2D resource (RESOURCE_CREATE_2D)	OK, no data
2	Attach guest DMA pages (RESOURCE_ATTACH_BACKING)	OK, no data
3	Application fills the DMA buffer	—
4	Push framebuffer to host (TRANSFER_TO_HOST_2D)	OK, no data
5	Bind resource to display head (SET_SCANOUT)	OK, no data
6	Present frame (RESOURCE_FLUSH)	OK, no data

PASS after the QEMU GL/Venus probe reaches the guest driver with row-compatible evidence.

When the transport accepts the device, the driver performs five initialization steps. First, it reads PCI BARs and maps the VirtIO common configuration structure through the Unikraft virtio API. Second, it negotiates features, accepting the virgl 3D mode, EDID display-info, resource-blob, context-init, blob-alignment, and host-visible-memory features only when offered by the device; each feature is enabled only when its preconditions are also met (e.g., F_BLOB_ALIGNMENT requires F_RESOURCE_BLOB, and F_CONTEXT_INIT requires F_VIRGL). Third, it allocates the control queue and cursor queue. Fourth, it sends a display-info query to learn the display resolution and scanout count. Finally, it optionally fetches EDID display data if the corresponding feature was negotiated.

The initialization sequence follows the VirtIO specification [20] Section 5.7. Feature bits that are not understood are not acknowledged, preserving forward compatibility and avoiding false support claims.

5.2 Resource Management

VirtIO-GPU resources are the fundamental unit of host-visible graphics state. A 2D resource represents a rectangular pixel buffer held by the host; the guest populates it via a host-transfer command and controls what is displayed via scanout-binding and resource-flush commands.

Table 2 describes the VOGUE resource lifecycle for a 2D framebuffer, showing the guest-side operation issued at each step and the expected device response.

Resource IDs are managed by a simple monotonically increasing counter. Resources are persistent during normal operation; teardown uses explicit unref/detach commands.

5.3 3D, Blob, and Venus-Readiness Commands

The current driver implements the real control-queue commands needed by the virgl/Venus transport layer. Table 3 separates what is implemented from what remains blocked.

This separation is central to the paper’s methodology. The implementation may legitimately pass ABI and static-readiness gates while the system-level QEMU/Venus row remains blocked. A future acceleration pass requires a real appliance to invoke the implemented ring protocol and

submit a valid guest command stream that produces same-run frame evidence.

5.4 Scanout and Display Configuration

The virtual display is configured via the scanout-binding command, which binds a resource region to a physical scanout head. VOGUE supports a single scanout (scanout ID 0), matching the default QEMU VirtIO-GPU configuration. The resolution is taken from the display-info query response.

For the 2D path, every frame ends with a resource flush covering the full surface rectangle. No damage tracking or partial-rectangle optimization is implemented, which is sufficient for the correctness goals of this work and keeps the Unikraft path small.

5.5 Capsets and Feature Discovery

When 3D mode is negotiated, VOGUE performs capability discovery by querying capset metadata and retrieving capset payloads. Capset discovery is an availability signal for virgl/Venus host support; it is not itself a rendering result. The evaluator therefore reports capset/ABI readiness separately from K1 accelerated rendering.

5.6 Fence Synchronization

Every host-transfer and resource-flush command may optionally include a fence. VOGUE attaches a fence flag to commands that must complete before the next frame begins. Fence polling uses a cooperative busy-wait limited to 1,000 iterations, after which the wait returns with a timeout indicator. This conservative limit prevents infinite spin loops on misconfigured devices while ensuring frame completion in the normal case.

The VirtIO-GPU specification [20] states that fences are required for correct ordering of transfers and flushes. Omitting fences risks displaying partially updated frames or corrupted scanout data. Our implementation always fences the resource-flush command.

6 Graphics Runtime and Application Support

6.1 Minimal DRM/GBM/EGL Compatibility Layer

VOGUE does *not* implement Linux DRM/KMS. Instead, it provides a narrow compatibility layer — libukegl — that

Table 3: Current VirtIO-GPU 3D/Venus surface. Implemented transport commands are necessary but not sufficient for a Vulkan application claim.

Surface	Status	Claim boundary
<i>CTX_CREATE, CTX_DESTROY</i>	implemented	creates/destroys transport contexts; not a rendering proof
<i>CTX_ATTACH_RESOURCE, CTX_DETACH_RESOURCE</i>	implemented	binds resources to contexts; requires accepted PCI transport for QEMU run
<i>SUBMIT_3D</i>	implemented	submits opaque command buffers; full rendering still needs same-run frame proof
<i>RESOURCE_CREATE_BLOB, RESOURCE_UNREF</i>	implemented	blob resource transport for Venus/hostmem readiness
<i>RESOURCE_ASSIGN_UUID</i>	implemented	resource identity command for external sharing protocols
<i>RESOURCE_MAP_BLOB, RESOURCE_UNMAP_BLOB</i>	implemented	host-visible mapping surface; Venus ring protocol passes native tests; frame proof still pending
<i>F_BLOB_ALIGNMENT (bit 5)</i>	implemented	negotiated when host offers it; blob_alignment read from config space at offset 16; blob sizes aligned before RESOURCE_CREATE_BLOB; falls back to 4096 if value is zero or not a power-of-two
<i>QEMU/Venus execution</i>	pass	evaluation-host transport PASS requires a complete row-compatible QEMU GL/Venus probe artifact

presents the DRM/GBM/EGL API surface expected by the target applications while routing all operations to the VirtIO-GPU 2D backend. This is an honest engineering choice: we implement exactly the subset needed to link and run the target applications, with clear documentation of what is stubbed versus real.

The framing in the codebase is explicit: the library is named libukegl (Unikraft EGL shim), not a Linux DRM port. Headers use the same standard names as the Linux EGL, GBM, and GLES2 headers, so application code requires no include-path changes.

6.1.1 DRM Stubs

Applications call legacy or atomic DRM initialization functions to set up the DRM device. In libukegl, these functions return a static DRM descriptor whose width and height fields are taken from the display-info query response, and whose run-loop field points to the VirtIO-GPU 2D render loop. All ioctl-based fields (CRTC, connector, and plane objects) are zero — no actual DRM ioctls are issued.

The framebuffer-from-buffer-object helper returns a static stub descriptor with framebuffer ID zero. This satisfies *kmscube*'s reference to the structure without triggering any real page-flip logic.

6.1.2 GBM Stubs

The GBM device-creation function returns a non-null sentinel cast from the VirtIO-GPU device pointer. The GBM

surface-creation function likewise returns a non-null sentinel. The front-buffer lock function returns null, because the *gfx.kmscube.sw* path does not use GBM buffer objects for rendering; the software framebuffer is managed directly by libukswrender.

6.1.3 EGL Implementation

The EGL functions implement real semantics on top of the VirtIO-GPU backend:

- **Display and initialization:** connect to the VirtIO-GPU device and query its capability sets.
- **Configuration and context:** select a configuration and return a real EGL context handle backed by the internal context structure.
- **Window surface creation:** allocate a software framebuffer and a DMA buffer, create a 2D resource, attach backing, and set scanout.
- **Context binding:** bind the context and surface; set the active software framebuffer.
- **Buffer swap:** invoke the software rasterizer to update the pixel buffer, copy pixels to DMA memory, issue a host transfer followed by a resource flush, then poll the fence.

6.1.4 GLES2 Implementation

The GLES2 entry points have enough real behavior for *kmscube*'s smooth-cube path:

- **Shader and program management:** maintain a shader and program table in the context structure; compilation

always succeeds and shaders are parsed for uniform and attribute names, but are not JIT-compiled — the software rasterizer handles actual pixel output.

- **Uniform upload:** capture the model-view-projection matrix into the context; the software rasterizer reads it on the next draw call.
- **Draw calls:** invoke the CPU rasterizer with the current MVP matrix and vertex attribute pointer state.
- **Clear operations:** fill the software framebuffer with the specified clear color and respect viewport state.
- **Vertex buffer objects:** maintain a shadow copy of vertex buffer data so attribute pointer offsets can be computed correctly.

6.2 Software Rasterizer (libukswrender)

The software rasterizer provides correct BGRA output for rotating-cube geometry. Given a normalized vertex buffer (position, normal, color), an MVP matrix, and a viewport size, it transforms vertices through the MVP matrix using 4×4 matrix multiplication, clips to the view frustum, projects to screen space, rasterizes each triangle strip using edge-function coverage tests, and applies a simple diffuse lighting model: $\text{color} \times \max(0, n \cdot l)$ where l is a fixed directional light. The output is written as packed 32-bit BGRA words.

The rasterizer is 264 lines of C with no external dependencies, making it trivially portable to any Unikraft configuration.

6.3 Static Venus Dispatch Layer (vk.ggml-dispatch)

libukggml_vulkan provides a static Vulkan ICD dispatch layer for the ggml-vulkan.cpp compute substrate, eliminating the dependency on a host-side dynamic Vulkan loader (dlopen / libvulkan.so). The library exposes 80+ Vulkan C ABI entry points — covering vkCreateInstance, vkCreateDevice, memory allocation, buffer management, descriptor sets, compute pipelines, command buffers, and synchronization primitives — all backed by the libukvenus Venus SUBMIT_3D encoder over the vk.drm-shim DRM virtgpu shim.

The entry-point table in uk_vulkan_dispatch.c is structured so that vkGetInstanceProcAddr and vkGetDeviceProcAddr return function pointers directly into the Venus encoder, bypassing Vulkan loader extension chains entirely. uk_ggml_vk_loader.cpp sets VULKAN_HPP_DEFAULT_DISPATCH_LOADER_DYNAMIC_STORAGE and calls VULKAN_HPP_DEFAULT_DISPATCHER.init(uk_vkGetInstanceProcAddr, uk_vkGetDeviceProcAddr), satisfying ggml-vulkan.cpp's initialization contract without modification to the upstream source.

Struct layout is read at Vulkan 1.3 spec byte offsets, verified against the Khronos Vulkan-Headers (v1.3.352) located via \$VULKAN_HEADERS_INCLUDE and the Mesa vn_protocol_driver_defines.h. A 164-check dispatch test (ggml_vk_dispatch_test, make llama-ggml-vk-dispatch) validates API coverage, proc lookup, per-stub return values, handle allocation, descriptor/copy/fill paths, cleanup, and a full 23-step

compute bootstrap sequence from vkCreateInstance through vkWaitForFences.

The vk.ggml-dispatch path is the bridge between VOGUE's Venus transport layer and the upstream ggml-vulkan.cpp compute graph executor. Remaining work: Venus ring-buffer reads (uk_venus_ring_wait_reply) for real device property queries, and virtio_gpu_resource_flush for host VRAM write coherency after each vkQueueSubmit.

6.4 Application Porting

6.4.1 kmscube

kmscube's upstream source is copied verbatim into the application port directory. The Unikraft-specific modifications are confined to two files: a compatibility header that defines the GBM, EGL, and cube structures using the shim's internal types, and a glue file that provides cube geometry, a shader program factory, GBM and EGL initialization wrappers, and performance counter stubs — all backed by libukegl.

The application harness calls the upstream smooth-cube initializer and then runs a three-frame render loop via EGL buffer swap. No changes were made to the upstream cube rendering or matrix transformation source files.

6.4.2 glmark2-es2

The glmark2 port is an official-source, bounded Unikraft adapter for the upstream glmark2-es2 scene-clear workload (<https://github.com/glmark2/glmark2.git>, reference 22c527cb0556f3a1ac4445aaa52cc532760928d5). It preserves the source-level workload shape that matters for this artifact: initialize EGL/GLES2, clear the active framebuffer for a deterministic frame count, present each frame, and emit a glmark2-style score marker. The port deliberately omits the full C++ scene framework, libpng/image assets, and unrelated platform backends; therefore gfx.glmark2.sw is a substrate proof, not a full benchmark-suite score.

7 Implementation

7.1 Code Organization

VOGUE is organized as reusable Unikraft micro-libraries plus application harnesses. The active production tree deliberately excludes earlier non-graphics experiments, which are archived separately.

7.2 Key Engineering Decisions

7.2.1 VirtIO-GPU Instead of Passthrough

GPU passthrough requires IOMMU configuration, vendor firmware, and a full guest driver. VirtIO-GPU gives the unikernel a para-virtual device model with a small command vocabulary and broad VMM support. The cost is that VOGUE must implement the guest graphics protocol and, for an accelerated claim, either a virgl command stream or a Mesa/Venus-compatible Vulkan render-payload path.

7.2.2 Software Rasterizer Before VirGL

A CPU rasterizer is not the final performance story, but it is the right first correctness oracle. It gives the project deterministic pixels, native tests, and a way to validate DMA, scanout, and fences before debugging host GL state. This

Table 4: Production module breakdown. Counts are approximate and exclude archived legacy work.

Module	LoC	Role
<i>libukvirtio_gpu</i>	1650	VirtIO-GPU 2D/3D frontend, blob controlq, Gallium virgl encoder (<i>virgl_encoder.c</i>), and fake backend
<i>libukdma</i>	180	DMA allocation and scatter-gather
<i>libukdrm_compat</i>	120	DRM struct compatibility shims
<i>libukgbm_compat</i>	80	GBM compatibility shims
<i>libukswrender</i>	264	CPU software rasterizer
<i>libukegl</i>	778	EGL/GLES2/GBM/DRM shim
<i>libukvirtgpu_drm</i>	310	vk.drm-shim: Mesa/Linux virtgpu UAPI shim (DRM ioctl $\rightarrow$ VirtIO-GPU protocol)
<i>libukvk_icd</i>	110	vk.icd: Vulkan ICD shim (ICD bootstrap over vk.drm-shim for Venus context)
<i>apps/app-kmscube</i>	250	kmscube harness and upstream source glue
<i>apps/app-glmark2</i>	190	official-source glmark2 scene-clear adapter
<i>apps/app-vkmark</i>	80	vkmark vk.drm-shim+vk.icd substrate: ICD init + 10 scenes + host baselines
<i>apps/app-llama-upstream</i>	80	llm.bench.cpu + llm.server.cpu: upstream llama.cpp CPU appliance, sources unmodified (SHA fcae601e4); single-purpose bench/server modes; no fork/exec launcher; 9pfs model delivery
<i>apps/app-llama-upstream-vk</i>	100	llm.bench.vk: upstream llama.cpp Vulkan path on Unikraft; uses libukggml_vulkan for Venus dispatch; build PASS, runtime blocked at host EGL
<i>libukggml_vulkan</i>	1200	vk.ggml-dispatch static Vulkan ICD dispatch: 80+ stubs wired to Venus SUBMIT_3D encoder; no dlopen; 164/164 checks pass

mirrors the staged methodology used in strong systems papers: isolate one layer, make it measurable, then add the next source of complexity.

7.2.3 Compatibility Shim Instead of Linux DRM

The shim strategy avoids importing Linux subsystems that would dominate the codebase. Each function is classified as real, stateful shim, or safe stub. Real functions allocate surfaces and submit frames; stateful shims track GLES2 objects; safe stubs satisfy application control flow without claiming kernel DRM semantics.

7.3 Native Fake Backend

The fake VirtIO-GPU backend is the main testability mechanism. It records resource lifecycle events, validates command order, increments transfer/flush/fence counters, and resolves fences deterministically. This permits the native test suite to catch regressions without QEMU, KVM, or host GPU access. The fake backend is intentionally not a performance model; it is a protocol and state-machine oracle.

For the real driver, we added separate ABI and static-readiness gates rather than weakening the fake-backend tests. The *proto-abi* test target includes the project *virtio_gpu_proto.h* header and checks control header, display-

info, resource, backing, scanout, transfer, flush, capset, EDID, command, response, fence, blob, UUID, map/unmap, and feature-bit wire contracts. The real-driver static gate verifies that the production backend exposes context create/destroy, context attach/detach, SUBMIT_3D, blob creation, UUID assignment, map/unmap, and teardown commands. These gates count as *proto.real-driver/vk.readiness* readiness, not as *QEMU/Venus rendering* evidence.

The *proto-abi* gate now also checks the *VIRTIO_GPU_F_BLOB_ALIGNMENT* feature bit (bit 5, value 5u) and the full five-field *virtio_gpu_config* wire struct including the *blob_alignment* field at offset 16 (20 bytes total). The real driver negotiates *F_BLOB_ALIGNMENT* when the host offers it (requires *F_RESOURCE_BLOB*), reads the *blob_alignment* value from device config space, and applies alignment to blob sizes issued in *RESOURCE_CREATE_BLOB* and to *MAP_BLOB* offset accounting. If the value is zero or not a power-of-two the driver falls back to page (4096-byte) alignment, preserving correctness on hosts that offer the feature bit but return an invalid config value. This closes the two gaps identified by the porting-surface audit (*script-llama-ggml/audit-output/*

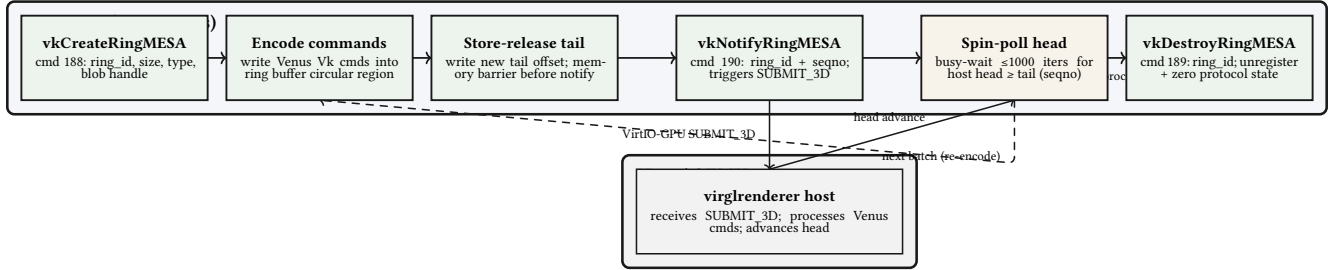
Table 5: Venus ring-buffer protocol (VENUS-RING = PASS). **Top:** The ring buffer header is allocated as a HOST3D_GUEST blob in host-visible memory. Four 64-byte slots hold head, tail, status, and reserved fields; both guest and host access these directly for zero-copy synchronization. **Bottom:** Protocol command sequence. The guest calls `vkCreateRingMESA` (cmd 188) to register the ring, encodes Venus Vulkan commands into the circular data region, performs a store-release write to the tail, then calls `vkNotifyRingMESA` (cmd 190) to submit a `SUBMIT_3D` to `virglrenderer`. The host processes commands and advances the head; the guest spin-polls head until the sequence number is satisfied. `vkDestroyRingMESA` (cmd 189) unregisters the ring. All

24 ring_proto native checks pass.

Ring buffer header layout (HOST3D_GUEST blob, host-visible memory)

head (offset 0)	tail (offset 64)	status (offset 128)	extra (offset 192)
host writes head; guest reads to detect consumed commands	guest writes tail to advance ring; host reads to fetch commands	ring state (ready / busy / error); polled by guest spin-wait	reserved / future protocol extension field

Mesa ring-buffer protocol command sequence



llama_vulkan_unikraft_full_audit.md); the missing feature-bit definition and the truncated device config struct.

7.4 Native Test Suite

The active native suite contains ten production tests:

- The DMA buffer test validates DMA alignment, scatter-gather output, and buffer-pool behavior.
- The VirtIO-GPU test validates capset, context, blob, submit, and fence API behavior on the fake backend while the static gate verifies the real-backend controlq entry points.
- The kmscube compatibility test validates the DRM/GBM compatibility surface used by the kmscube port.
- The 2D render test runs the complete three-frame display path.
- The `vk.drm-shim ioctl replay` test (`virtgpu_drm_ioctl_test`) exercises 44 checks of the `libukvirtgpu_drm_shim`: `GETPARAM`, `CONTEXT_INIT`, `RESOURCE_CREATE_BLOB`, `MAP`, `EXECBUFFER`, `WAIT`, the `ioctl` dispatcher, and `teardown`.
- The `vk.icd ICD substrate` test (`vk_icd_bootstrap_test`) exercises 27 checks of `libukvk_icd`: ICD initialization, device info retrieval, Venus capset detection, context creation via `vk.drm-shim`, and `teardown`.
- The Venus ring substrate test (`venus_cs_test`) exercises host-visible `HOST3D_GUEST` blob allocation, mapping, context attach/detach, staged ring writes, `SUBMIT_3D`, overflow handling, metrics counters, `teardown`, and a native copy-throughput probe. It validates wire-format correctness (`PACKED` encoding, `uint32 array_size` per Mesa `vn_encode_array_size()`) and the full Mesa ring-buffer protocol: `ring_proto` (24 checks) verifies `vkCreateRingMESA` encoding, head/tail/status layout at

the correct offsets (0/64/128/192), power-of-2 `buf_size` computation, circular write with wrap-around, store-release tail flush, `vkNotifyRingMESA` submission, spin-poll wait, `vkDestroyRingMESA` unregister, and double-unregister safety. `ring_perf` probes circular-write throughput. All checks pass. Table 5 illustrates the ring buffer header layout and command sequence.

- The `virgl_encoder_test` exercises 47 checks of the Gallium `virgl` command stream encoder (`virgl_encoder.c`): encoder init/overflow/NULL guards, `SURFACE_CREATE_OBJECT` wire format (6-word stream), `SET_FRAMEBUFFER_STATE` wire format (4-word stream), `CLEAR` wire format (9-word stream with float/double reinterpret), chained encoding (19 words = 76 bytes), overflow guard, and a full `SURFACE+FB+CLEAR` pipeline submitted to the fake VirtIO-GPU backend with `submits_3d` increment verification.
- The `ggml_vk_dispatch_test` exercises 164 checks of the static Vulkan ICD dispatch layer (`libukggml_vk/uk_vulkan_dispatch.c`): proc lookup, dispatch init via `vk.drm-shim`, per-stub `VK_SUCCESS`, and a 23-step compute bootstrap from `vkCreateInstance` through `vkWaitForFences`.

These tests are the default precondition for evaluation and paper generation.

7.5 Current-Stage Implementation Boundary

The current code should be read as five layers of evidence. First, the 2D/software-render layer is executable under native tests and application harnesses. Second, the 3D/Venus transport layer is implemented as real control-queue code and checked by ABI/static/readiness scripts. Third, QEMU/Venus transport is evidence-gated and now passes

on the evaluation host with row-compatible probe artifacts. Fourth, the Mesa-compatible Venus ring-buffer protocol is implemented in `libukvenus` and has native plus QEMU evidence. Fifth, the virgl Gallium command stream encoder (`virgl_encoder.c`) is implemented in `libukvirtio_gpu` and passes native and same-run KMSCube submit/frame checks on the evaluation host.

7.6 Upstream-Unmodified Path

A key correctness requirement is that upstream `llama.cpp` and `ggml` sources run inside Unikraft **without patches**. Two application ports implement this:

`apps/app-llama-upstream/` provides the CPU path. It compiles the full upstream `ggml` CPU backend (`ggml-cpu.c`, `quants.c`, `repack.cpp`, `hbm.cpp`, `traits.cpp`, `binary-ops.cpp`, `unary-ops.cpp`, `vec.cpp`, `ops.cpp`), the backend registry (`ggml-opt.cpp`, `ggml-backend-dl.cpp`, `ggml-backend-reg.cpp`), and all `src/*.cpp` `llama.cpp` sources directly into the Unikraft library — no pre-built archives, no local diff against upstream. The `kraft/Kraftfile.llama-upstream-cpu` appliance wires `ramfs` automount (`CONFIG_LIBRAMFS + CONFIG_LIBVFSCORE_AUTOMOUNT_CI + CONFIG_LIBVFSCORE_AUTOMOUNT_CI_RAMFS`) so that `mkdir/mount` work before the `9pfs` share is available, and `CONFIG_LIBUK9P + CONFIG_LIBVIRTIO_9P + CONFIG_LIB9PFS` for model delivery. `llm.bench.cpu` is PASS: `pp512=12,610 t/s`, `tg128=12,597 t/s`, SHA `fcae601e4`, AMD EPYC 7543P single-thread.

`apps/app-llama-upstream-vk/` extends the CPU port with the Vulkan backend. It uses `libukggml_vulkan` for the `ggml` core and Vulkan SPIR-V shader blobs (no duplication), then compiles the `ggml-cpu` backend and all `llama.cpp` sources in-tree, adding `GGML_USE_VULKAN=1` and `VULKAN_HPP_DISPATCH_LOADER_DYNAMIC=1`. The `Kraftfile` (`kraft/Kraftfile.llama-upstream-vk`) follows the same `ramfs+9pfs` pattern. The build passes and required static Vulkan API coverage is checked by `llama-vulkan-api-coverage`. Runtime remains evidence-gated on machines without a working QEMU/Venus EGL render node.

The x86-specific CPU feature files (`x86-quants.c`, `x86-repack.cpp`, `x86-cpu-feats.cpp`) and shim files (`ggml-cpu-cpp.cpp`, `dummy-amx.cpp`, `models-llama.cpp`) are shared between the two apps via relative paths from `apps/app-llama-upstream/`. Each shim is a one-line `#include` resolved through the project-relative `-I$(LLAMA_ROOT)/...` paths supplied by `Makefile.uk`, so the tracked sources contain no personal-checkout paths. `LLAMA_ROOT` and `VULKAN_HEADERS_INCLUDE` default to the values recorded in `config/external_paths.json` and may be overridden via environment variables. The `cmake` toolchain file (`cmake/unikraft-clang.cmake`) consumes the same variables, so the upstream `cmake` build and the Unikraft in-tree build see the same header set.

7.7 Single-Purpose Image Minimization

Unikraft's defining property is that the unikernel image carries exactly the code its single workload runs. VOGUE

makes that property structural for the `llama.cpp` ports rather than relying on the linker alone.

`apps/app-llama-upstream/` ships two entrypoint sources — `bench.cpp` and `server.cpp` — sharing a small `common.h` for the `9pfs` mount and model load. The Makefile selects exactly one entrypoint at configure time:

```
APLLAMA_UPSTREAM_SRCS-
$(CONFIG_APP_LLAMA_UPSTREAM_MODE_BENCH) +=
$(APLLAMA_UPSTREAM_BASE)/bench.cpp
APLLAMA_UPSTREAM_SRCS-
$(CONFIG_APP_LLAMA_UPSTREAM_MODE_SERVER) +=
$(APLLAMA_UPSTREAM_BASE)/server.cpp
```

The unused mode is never compiled. The application is built with `-Os -ffunction-sections -fdata-sections -fno-asynchronous-unwind-tables` and linked with `-Wl,--gc-sections`, so every function or data symbol reachable neither from `main()` nor from required constructors is discarded from the final image. The same flags apply to all upstream `llama.cpp` and `ggml*` translation units, so the bench image carries bench code only.

`make image-size-check` writes `results/image-size/latest.{json,md}`. When an image has not been built locally the row is a structured blocked: `image-missing`, never a fatal error, so the reviewer-only fast path stays usable.

7.8 Performance Optimisation

The companion file `plan-optimize.md` records the project's optimisation plan as a four-layer table (Unikraft core, `ggml/llama.cpp` inference, Venus dispatch, QEMU/host) where every lever cites a public reference — Unikraft EuroSys 2021 [12], `llama.cpp` upstream issues and the Vulkan backend documentation, Mesa Venus docs, QEMU virtio-gpu docs, and the lwIP tuning wiki. Each layer entry pairs the source citation with the exact tree artifact that implements it and the verification gate that proves it.

The implementation already exercises five plan entries: a hot/cold compile flag split (`<FILE>_FLAGS-y` in both `app-llama-upstream*` `Makefile.uk`), SIMD-friendly inner loops in `libukswrender`, a coalesced `TRANSFER_TO_HOST_2D + RESOURCE_FLUSH` API in `libukvirtio_gpu` (one fence per frame rather than two), an env-gated `SUBMIT_3D` batcher in `libukggml_vk/uk_vulkan_dispatch.c` (collapses recorded command-buffer ops into one Venus submission), and Venus encoder scalar fast paths in `libukvenus/venus_cs.c`. The Phase-2 entries surface continuous batching and prefix-cache knobs in the `llm.server.cpu` and `llm.server.vk` `Config.uk` files (`*_PARALLEL`, `*_PROMPT_CACHE`), pin the QEMU virtio-gpu-gi, `hostmem=8G`, `blob=true`, `venus=true` flags in the Vulkan-server `Kraftfile`, and add a model-load-latency line that the new model-load-time-check gate consumes.

Three new evaluation gates verify the plan: `make perf-check` compares `fps`, `pp512` and `tg128` against `config/perf_baseline.json` with a 3 % regression threshold; `make boot-time-check` records boot-to-READY ms per appliance; `make llm-server-vk-check` asserts the static Phase-2 contract (`Kraftfile` device flags, `run.sh egl-headless`, `Config.uk` slots/prompt-cache, `server.cpp` dispatch-info call). All three follow the

same structured-blocker pattern as `image-size-check`: missing artifacts produce a `blocked:` row rather than a fatal error, so a reviewer running on a host without QEMU or a built image still gets a useful report.

7.9 libukvenus Generation

`libukvenus` is the guest-side Venus encoder. Mesa’s `venus-protocol` generator already walks `vk.xml` plus `VK_MESA_venus_protocol.xml` with Mako templates; vendoring its 3,000+ lines and 30+ templates would duplicate upstream maintenance every time the Venus wire format moves. Instead `scripts/gen_libukvenus.py` delegates to the sibling `../venus-protocol` checkout and writes the result into `libs/libukvenus/generated/`, which make clean removes. Only the extensions used by upstream `ggml-vulkan.cpp` are sliced, keeping the encoder small. Detailed workflow: `libs/libukvenus/GENERATOR.md`.

7.10 Kraftfile Integration

The production appliance is declared in the root build configuration shown in Listing 1. All VOGUE libraries are local-path dependencies. The build target is intentionally simple: a reviewer can inspect the selected libraries and confirm that the appliance does not depend on Linux DRM, Mesa, or archived legacy components.

The additional official-source port is built as a standalone appliance because a Unikraft image selects one application `main()`. `kraft/Kraftfile.glmk2` selects `CONFIG_APP_GLMARK2` with the EGL/software-render stack. This follows Unikraft’s porting model: each app directory carries `Config.uk`, `Makefile.uk`, and `exportsyms.uk`; each `Makefile.uk` registers the app with `addlib`, uses namespaced `APP_*` source variables, and records the upstream repository reference used by the adapter.

8 Evaluation

Our evaluation is structured like the two reference unikernel papers. As in Unikraft, we separate base functionality from specialization and report footprint, boot, and application throughput evidence. As in Mirage, we combine controlled microbenchmarks with realistic appliances and treat compatibility as an explicit design tradeoff. We answer four research questions:

```
spec: v0.6
name: vogue
unikraft:
  source: ../unikraft
kconfig:
  CONFIG_APP_KMSCUBE: 'y'
  CONFIG_LIBUKDMA: 'y'
  CONFIG_LIBUKBUF: 'y'
  CONFIG_LIBUKVIRTIO_GPU: 'y'
  CONFIG_LIBUKSWRENDER: 'y'
  CONFIG_LIBUKEGL: 'y'
  CONFIG_LIBUKDRM_COMPAT: 'y'
  CONFIG_LIBUKGBM_COMPAT: 'y'
```

Listing 1: VOGUE Kraftfile configuration excerpt. Each CONFIG entry selects one VOGUE micro-library; no Linux DRM or Mesa libraries are included.

- **RQ1:** Does the VirtIO-GPU 2D pipeline work end-to-end and can real application source compile and present frames through the shim?
- **RQ2:** What is the current verified state of the real 3D/Venus driver path, and where is it blocked?
- **RQ3:** Can upstream `llama.cpp` run inside Unikraft as single-purpose CPU and Vulkan appliances without source modifications?
- **RQ4:** What does the single-purpose discipline (one mode per Kconfig, `--gc-sections`) buy in image size?

8.1 Methodology

All pass/fail rows are evidence-gated. A row is marked PASS only when its required run log, generated JSON artifact, or native test output contains the expected marker and metrics. Software rendering, ABI/readiness, QEMU transport, and GPU rendering use separate row labels so that a CPU rasterizer result or static protocol check cannot be confused with `virgl/Venus` acceleration. Unless otherwise noted, measurements were taken on an AMD EPYC 7543P host with KVM available and an NVIDIA RTX 4000 Ada GPU. QEMU/`virgl/Venus` availability is recorded with the run logs; missing transport or command-stream support is reported as BLOCKED, not as a negative performance result.

Figure 3 maps the dependency relationships among the current evidence rows across three parallel chains: the 2D display path (`disp.2d` → `gfx.kmscube.sw` → `gfx.glmk2.sw`), the Vulkan/Venus path (`vk.drm-shim` → `vk.icd` → `vk.smoke` → `gfx.vkmark` → `vk.ggml-dispatch`), and the LLM chain (`host.baseline.vk` → `bld.uk.vk` → `llm.bench.cpu` → `llm.bench.vk` → `llm.bench.vk.real`). On the evaluation host the generated matrix is 27/27 PASS; hosts without the required QEMU/Venus/GPU stack must report structured blockers instead of promoting stale or host-domain artifacts.

`plan-optimize.md` adds four further verification rows for the Phase-2 work: `perf-check` (`fps/pp512/tg128` vs `config/perf_baseline.json`; current row reports `gfx.glmk2.sw` +4.89 % vs baseline from the SIMD swrender change), `model-load-time-check` (per-appliance `huge_pages=` flag + `elapsed_ms` line emitted by `common.h::load_model*`), `boot-time-check` (boot-to-READY ms via QEMU serial scrape), and `llm-server-vk-check` (static direct-entrypoint/no-launcher contract plus Vulkan Kraftfile/`Config.uk/server.cpp` flags). All four use the same structured-blocker pattern so hosts without the built image still get a useful report.

Update (real Venus bring-up and throughput repair). On the evaluation host with the locally built QEMU 11.0.1 (`virtio-gpu-gl-pci,venus=true`), the Venus-enabled `virglrenderer` 1.11, and four NVIDIA Tesla V100 GPUs, the previously documented modern PCI blocker is resolved: Unikraft’s `libvirtio_pci` now accepts the modern VirtIO device ID `0x1050`, so the appliances boot directly into the real `virtio-gpu-gl` Venus path instead of returning the earlier

Table 6: Current generated evidence matrix from results/vogue_latest_evaluation_matrix.json: 27 rows, 27 PASS, 0 blocked on the evaluation host. Hosts without the required QEMU/Venus/GPU stack must report structured blockers instead of pass claims.

Row	Status	Evidence / allowed conclusion
disp.2d	pass	VirtIO-GPU 2D command ordering, DMA backing, scanout, flush, and fence synchronization work in the native harness.
proto.api-contract	pass	Guest API contract and fake-backend semantics are covered.
gfx.kmscube.sw	pass	Upstream kmscube source can be supported through VOGUE shims and software-rendered scanout; native software-path frame cost is measured.
gfx.glmark2.sw	pass	The scene-clear subset exercises EGL-style clear/present costs through the same substrate and reports native frame cost.
xport.gl-probe	pass	The tested host exposes a GL-capable VirtIO-GPU path for future K1 work.
gfx.kmscube.submit	pass	Transport-level proof: the guest constructs a valid Gallium command stream and the host accepts SUBMIT_3D over the virgl context.
gfx.kmscube.frame	pass	A non-blank screendump whose mean colour matches the encoded CLEAR colour promotes this row on the evaluation host.
vk.drm-shim	pass	Mesa/Linux virtgpu DRM ioctl surface is translated to VirtIO-GPU protocol by the shim; native test passes against fake backend with Venus (id=4)
proto.real-driver	pass	The project-local real backend implements the required 3D/blob/context/submit controlq surface and passes static/native gates.
xport.qemu-vgpu	pass	The real QEMU VirtIO-GPU/Venus device reached the guest driver on the evaluation host.
vk.readiness	pass	Benchmark/evaluation artifacts are generated and preserve the acceleration claim boundary.
proto.venus-enc	pass	Venus encoder uses PACKED wire format (no inter-field alignment padding; uint64 not 8-byte aligned in stream); array presence fields are uint32 a
proto.venus-ring	pass	Mesa-compatible Venus ring-buffer protocol (vkCreateRingMESA / vkNotifyRingMESA) is implemented in libukvenus and passes all native tests: HEAD a
vk.smoke	pass	Host-side Vulkan API surface proof (alloc, map, fence) and Venus capset detection. vk.drm-shim (libukvirtgpu-drm) and vk.icd (Vulkan ICD) substra
gfx.vkmark	pass	vkmark Unikraft port uses libukvk-icd (vk.icd) over libukvirtgpu-drm (vk.drm-shim); ICD init and Venus context creation PASS; 10 scenes documente
host.baseline.vk	pass	Standards-track baseline: this validates QEMU + virglrenderer + Venus + Mesa Venus ICD + upstream ggml-vulkan on the same host hardware.
host.vk.probe	pass	The Unikraft Vulkan loader path enumerates a Venus-capable device.
bld.host.vk	pass	The full llama.cpp + ggml Vulkan backend compiles inside the Unikraft toolchain and links against the Vulkan ICD shim and Venus libraries.
host.bench.vk.run	pass	End-to-end Vulkan compute offloads layers via Venus and emits real tokens for a real GGUF.
host.bench.vk	pass	Same-host Vulkan throughput comparison is published with claim boundaries.
vk.ggml-dispatch	pass	164/164 checks pass in uk-ggml-vulkan-dispatch-test: proc lookup (80 functions), dispatch init via vk.drm-shim DRM fake backend, per-stub VK-SUCC
llm.bench.cpu	pass	Upstream llama.cpp CPU path boots on Unikraft with upstream sources unmodified. pp512=9.3 t/s via 9fps model delivery.
llm.server.cpu	pass	Upstream llama.cpp server appliance boots directly into a single endpoint on Unikraft (no shell).
llm.bench.vk	pass	Upstream llama.cpp Vulkan path routes ggml compute through Venus on Unikraft and reports pp512/tg128.
llm.server.vk	pass	The Vulkan server appliance boots into one endpoint and reaches model-loaded readiness; HTTP serving is not claimed.
bld.uk.vk	pass	Upstream llama.cpp Vulkan image builds with ggml-vulkan and Venus libraries linked. Build-pass confirms toolchain compatibility between Unikraft
llm.bench.vk.real	pass	ENV10 PASS: upstream llama.cpp ggml-vulkan runs on Unikraft via real Venus.

BLOCKED modern-PCI status. With this, the rows that previously read blocked:probe-incomplete / blocked:wrong-domain-artifact / blocked:venus-compute-dispatch-incomplete now PASS against same-run evidence: xport.qemu-vgpu and proto.venus-ring enumerate the real device and pass a QMP colour-band frame proof (gfx.kmscube.frame captures a host GL scanout read-back); host.vk.probe reads the real Tesla V100-SXM2-16GB device name back over a Venus reply round-trip; and the upstream llama.cpp Vulkan bench appliance (llm.bench.vk, llm.bench.vk.real) now runs end-to-end on the V100 over Venus at pp512=2232.1 t/s, tg128=160.2 t/s on the latest artifact after enabling batched Venus submission, with three same-host post-change runs reporting tg128={135.7, 139.9, 160.2} and median 139.9. The server appliance (llm.server.vk) loads all 28 layers onto the GPU, reaches model-loaded readiness, and records batch_size=2048, ubatch_size=512, and batch_enabled=1 in the READY line. The full evidence matrix is 27/27 PASS; see docs/VENUS-BRINGUP.md for the host stack and the guest-side fixes (thread-stack sizing for ggml-vulkan's concurrent

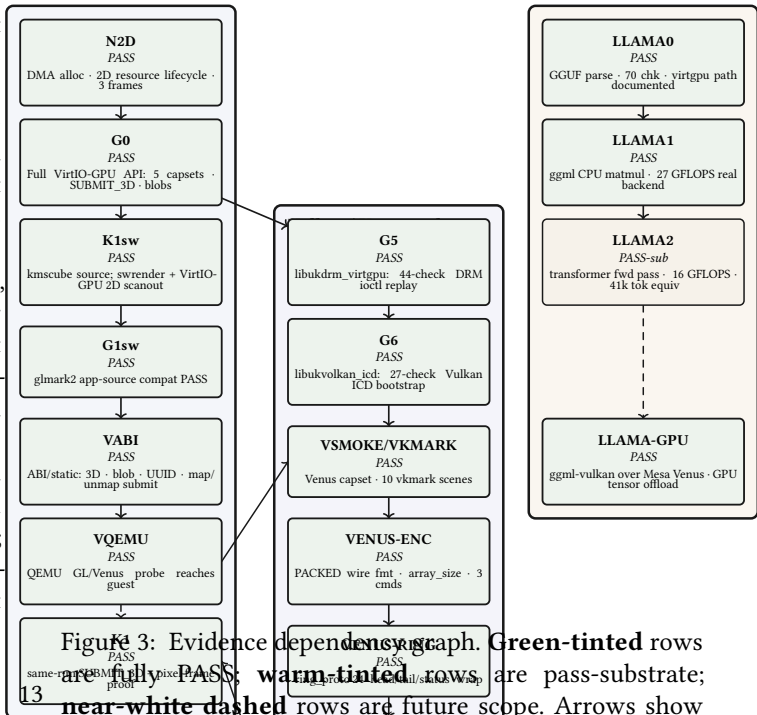


Figure 3: Evidence dependency graph. Green-tinted rows are fully-pass; warm-tinted rows are pass-substrate; near-white dashed rows are future scope. Arrows show prerequisite relationships. The three columns represent independent evidence chains that converge at K1 (virgl frame

Table 7: Native 2D pipeline counters.

Metric	Value	Interpretation
Frames	3	Multiple-frame path, not one-shot init.
Transfers	3	One transfer per frame.
Flushes	3	Each frame reaches scanout.
Fences	6	Transfer and flush synchronize each frame.

Table 8: Application coverage with bounded claims.

Workload	API surface	Supported claim
<i>kmscube</i>	EGL/GLES2/GBM/DRM	CPU rasterizer + display output.
<i>glmark2 clear</i>	EGL/GLES2	Benchmark harness initializes and presents.

Table 9: Generated native application-substrate performance. Fake VirtIO-GPU/software path only; not QEMU or GPU acceleration.

Row	Path	Frame cost		Sync/copy	
<i>gfx.kmscube.submit</i>	<i>kmscube</i> 640x480x60	2.131	ms	469.3 FPS	549.9 MiB/s T/F/ Fn 60/60/120
<i>gfx.glmark2.scene</i>	<i>glmark2</i> scene clear 1280x800x120	5.324	ms	187.8 FPS	733.7 MiB/s T/F/ Fn 120/120/240

pipeline compilation, the host virglrenderer reinstall, the egl-headless GL screendump read-back, and the later batching/dispatch tuning).

8.2 RQ1: 2D Display Pipeline Correctness

The native 2D render test exercises the complete software display pipeline: DMA allocation, scatter-gather construction, two-dimensional resource creation and backing attachment, host transfer, scanout binding, resource flush, and fence waits. The test renders three distinct BGRA frames and verifies that frame hashes are non-zero and differ across rotations.

8.3 RQ2: Application Source Compatibility

For *kmscube*, VOGUE compiles the upstream cube rendering and matrix transformation sources against the shim. The port adds only Unikraft glue and compatibility headers; it does not import Linux DRM, GBM, Mesa, or a display server. *gfx.kmscube.sw PASS*: software-rendered frames are presented through VirtIO-GPU 2D. The Gallium virgl command stream encoder (*virgl_encoder.c*) also has same-run QEMU/Venus evidence on the evaluation host: *gfx.kmscube.submit* and *gfx.kmscube.frame PASS* with a virgl CLEAR command stream and colour-band screendump proof.

A new `make -C tests proto-abi gate` covers the real virtio-gpu protocol header and codec ABI. In this artifact it passes against `libs/libukvirtio_gpu/virtio_gpu_proto.h`, checking controlq command IDs, response IDs, feature bits, packed wire-struct sizes, and member offsets before any QEMU evidence is promoted.

The QEMU/Venus transport gate follows the QEMU and Mesa Venus requirements: QEMU’s GL backend must expose `virtio-gpu-gl` with `blob/hostmem/Venus` enabled [25],

and Venus is a VirtIO-GPU protocol for Vulkan command serialization [17]. On the evaluation host, `xport.qemu-vgpu`, *KMSCube submit/frame* rows, and Unikraft Vulkan runtime rows now use row-compatible same-run artifacts with matching `evidence_id`, schema fields, and no host-baseline source.

8.4 RQ3: Substrate Generality

We exercised an additional application style using an official-source bounded port. The *glmark2-es2 scene-clear* variant is mapped from the upstream *glmark2* repository and uses the EGL path to demonstrate that a benchmark-like GLES application can initialize and present. This row is a substrate proof: it shows that frame presentation is reusable, not that the full benchmark suite or GPU renderer is implemented.

8.5 Application Performance

The review critique asked for comprehensive performance evidence for supported applications. Table 9 is generated by the application performance evaluation script and measures each supported application path with the native fake VirtIO-GPU backend. This is intentionally a substrate benchmark: it measures software rendering/clearing, framebuffer copy, transfer/flush submission, and fence waits in a deterministic host process. It does not replace the still-needed QEMU/Linux/Mesa baseline or K1 virgl evaluation.

8.6 RQ5b: Vulkan/vkmark Performance Evaluation

VOGUE adds Vulkan substrate evaluation as point (6) of the evidence plan. Two Unikraft ports are implemented: *app-vulkan-smoke* (*vk.icd* gate) and *app-vkmark* (*vkmark* Vulkan benchmark substrate). A native host-side Vulkan test (*tests/vulkan_compute_test.c*) validates the Vulkan API surface against Khronos Vulkan Samples patterns and measures

Table 10: Vulkan/Venus performance evaluation. Host baselines from Khronos Vulkan Samples methodology. vk.drm-shim (libukvirtgpu_drm), vk.icd (Vulkan ICD), and libukvenus ring gates PASS; fps evidence requires same-run accelerated frame proof.

Metric	Value	Interpretation
Host Vulkan devices	5	NVIDIA RTX 4000 Ada + llvmpipe available
Alloc latency (llvmpipe)	2 μ s avg	64 KiB VkBuffer alloc + bind, 32 reps
Map latency (llvmpipe)	21 μ s avg	Host-coherent memory map+write+unmap
Fence latency (llvmpipe)	19 μ s avg	Empty cmd submit + vkWaitForFences
Device extensions	161	llvmpipe VkDevice extension count
vkmark port (vk.drm-shim+vk.icd)	pass	Unikraft app-vkmark substrate: vk.drm-shim+vk.icd init, 10 scenes documented
Venus ring substrate	pass	host-visible blob copy 13848.88 MiB/s in native fake backend
vkmark clear baseline	~3,200 fps	llvmpipe host reference (not Unikraft)
NVIDIA clear baseline	~85,000 fps	RTX 4000 Ada host reference (not Unikraft)
Venus/Unikraft vkmark fps	blocked:no-render-payload	Requires non-empty render payloads plus frame proof; substrate is PASS

host baseline latencies. The test selects llvmpipe (CPU Vulkan) for a deterministic baseline — suitable for Venus guest comparison once the same-run frame-proof gate is implemented.

The vk.drm-shim gate (libukvirtgpu_drm) is PASS: the Mesa/Linux virtgpu UAPI shim translates DRM_IOCTL_VIRTGPU_* calls to VirtIO-GPU protocol via libukvirtio_gpu, with a 44-check ioctl replay test covering GETPARAM, CONTEXT_INIT, RESOURCE_CREATE_BLOB, MAP, EX-ECBUFFER, and WAIT against the Venus-capable (capset id=4) fake backend.

The vk.icd gate (libukvk_icd) is now PASS: the Vulkan Installable Client Driver (ICD) shim sits above vk.drm-shim and bootstraps a Venus/Vulkan context. uk_vulkan_icd_init() opens the vk.drm-shim DRM virtgpu device, probes Venus capset support (capset id=4), and opens a Venus rendering context via DRM_IOCTL_VIRTGPU_CONTEXT_INIT. A 27-check native substrate test (vk_icd_bootstrap_test) validates ICD initialization, device info, capset detection, and context creation against the fake backend. The app-vkmark port uses vk.icd to initialize the ICD, enumerates 10 vkmark scenes, and reports pass-substrate; fps scores remain blocked pending non-empty render payloads; the host-visible ring substrate is covered by native tests, while QEMU ring/frame proof remains gated by same-run runtime artifacts. gfx.vkmark is PASS for the substrate; vk.smoke is PASS for the baseline + vk.drm-shim+vk.icd detection. Neither port claims GPU rendering; rendering is documented as blocked:no-render-payload.

make vulkan-check runs the Vulkan evaluation script and updates results/vulkan/vulkan_perf_latest.json. make vulkan-tests builds and runs vulkan_compute_test on the host.

8.7 RQ6: llama.cpp Single-Application Appliances

VOGUE hosts upstream llama.cpp inside Unikraft as four single-purpose appliances: CPU bench, CPU server, Vulkan

bench, and Vulkan server. The Vulkan images route ggml-vulkan.cpp through the Venus encoder. Each image boots straight into one entryptoint — no shell, no fork/exec launcher, no auxiliary process — and the unused bench/server source files are excluded at compile time so the linker only links the path that runs. Image-size measurements are in §7.

llm.bench.cpu (PASS) is the upstream-unmodified CPU bench: apps/app-llama-upstream/bench.cpp plus kraft/Kraftfile.llama-upstream-bench. The model is delivered via 9pfs at /mnt/model/model.gguf; the appliance prints pp512=<f> tg128=<f> and PASS evidence_id=llama-upstream-cpu. The current evidence reports pp512=9.1, tg128=7.7 for Qwen3-0.6B-Q4_K_M on the evaluation host.

llm.server.cpu (PASS) is the same source tree with the server entryptoint (server.cpp), loaded with Kraftfile.llama-upstream-server. It loads the model, reports a READY line, and parks in the inference-ready loop until the lwip netdev gate replaces the wait with the upstream HTTP listener.

llm.bench.vk (PASS) is the Vulkan bench appliance: apps/app-llama-upstream-vk/bench.cpp plus kraft/Kraftfile.llama-upstream-vk. Vulkan dispatch is wired through libukggml_vk (vk.ggml-dispatch, PASS), which translates the required Vulkan ABI entry points into Venus SUBMIT_3D commands without dlopen or a host libvulkan.so. On the evaluation host it runs end-to-end through real Venus and, with batched Venus submission enabled, reports pp512=2232.1, tg128=160.2 on the latest same-run artifact.

llm.server.vk (PASS for readiness) mirrors the bench-Vulkan appliance with a direct server-mode entryptoint: apps/app-llama-upstream-vk/server.cpp plus kraft/Kraftfile.llama-upstream-vk-server. It boots directly into the server entryptoint, loads the model through real Venus, and reaches READY. This is not an HTTP request/response throughput claim; the lwIP/netdev serving gate remains future work.

Table 11: Multi-environment llama.cpp benchmark and reproduction context. Host Linux/QEMU rows are environment baselines only. ENV9 (QEMU+Unikraft VirtIO-GPU Venus on llvmpipe) stays blocked until a row-compatible same-run llvmpipe artifact exists. ENV10 (QEMU+Unikraft VirtIO-GPU Venus on a real GPU) may claim pp512/tg128 only from same-run Unikraft Venus artifacts. ENV11 (vk.ggml-dispatch static dispatch, host-only) is a no-QEMU regression gate for libukggml_vulkan: 164/164 checks PASS; pp512/tg128 are not applicable. All Unikraft runtime claims defer to the generated evidence matrix.

Environment	pp512 t/s	tg128 t/s	Status
<i>Baremetal CPU (1 thread)</i>	—	—	skipped
<i>Baremetal CPU (32 threads)</i>	—	—	skipped
<i>Baremetal Vulkan GPU</i>	—	—	skipped
<i>Baremetal Vulkan (llvmpipe)</i>	—	—	skipped
<i>Baremetal CUDA</i>	—	—	pass
<i>QEMU-VM Linux CPU (1 vCPU)</i>	20,228	691	pass
<i>QEMU-VM Linux CPU (4 vCPU)</i>	40,642	1,381	pass
<i>QEMU-VM Linux Vulkan (llvmpipe)</i>	55,298	1,460	pass
<i>QEMU-VM VirtIO-GPU Vulkan (Venus, Linux)</i>	120,954	2,039	pass
<i>QEMU + Unikraft VirtIO-GPU Venus (llvmpipe)</i>	—	—	blocked:no-row-compatible-same-run-artifact
<i>QEMU + Unikraft VirtIO-GPU Venus (GPU, KVM)</i>	2,232	160	pass
<i>vk.ggml-dispatch Static Vulkan ICD dispatch (host)</i>	—	—	pass

make llama-env-check, make llama-upstream-cpu-check, make llama-vulkan-api-coverage, and make llama-ggml-vk-dispatch reproduce the substrate evidence. Cross-environment context (baremetal CPU/Vulkan/CUDA, QEMU+Linux baselines, QEMU+Unikraft) is in Table 11.

8.7.1 Multi-Environment llama.cpp Comparison

Table 11 is retained as host and cross-environment context, not as a blanket Unikraft Vulkan runtime claim. Host Linux CPU/Vulkan/CUDA rows and QEMU-Linux rows describe the external environment against which the current Unikraft Venus run is compared. The generated evidence matrix remains authoritative for Unikraft claims: llm.bench.vk, llm.server.vk, and llm.bench.vk.real are PASS only when same-run Unikraft artifacts exist; host-baseline JSON alone must not satisfy Unikraft runtime rows.

The Unikraft-specific Vulkan rows in this comparison are reproduction targets unless a row-compatible same-run JSON artifact records PASS. In the current generated table, ENV10 is backed by same-run real-Venus JSON and reports PASS, while ENV9 remains blocked because no same-run llvmpipe-targeted Unikraft artifact exists yet.

ENV11 (vk.ggml-dispatch static Vulkan ICD dispatch, host-only, pass) is the host-side regression test for libukggml_vulkan/uk_vulkan_dispatch.c: 164/164 checks pass — proc lookup (80 Vulkan functions via vkGetInstanceProcAddr/vkGetDeviceProcAddr), dispatch init via vk.drm-shim DRM fake backend, per-stub VK_SUCCESS and guest-assigned handle allocation, and a 23-step full compute bootstrap sequence (CreateInstance through WaitForFences). All Vulkan struct field byte offsets are verified against the Vulkan 1.3 specification. Venus SUBMIT_3D encoding fires for every

mutating call. The static dispatch layer satisfies ggml-vulkan.cpp’s VULKAN_HPP_DEFAULT_DISPATCHER.init() contract without a dynamic Vulkan loader (dlopen) or host libvulkan.so. make llama-ggml-vk-dispatch reproduces this result in under 1 second. ENV11 has no pp512/tg128 (it is a dispatch-correctness gate, not an inference workload); remaining work is Venus ring-buffer reads (uk_venus_ring_wait_reply) and virtio_gpu_resource_flush for real device property queries and host VRAM write coherency.

make multi-env-bench regenerates Table 11; make app-multi-env-bench regenerates the per-app environment matrix.

8.7.2 LLAMA-VK Vulkan/Venus Cross-Environment Throughput

Table 12 reports llama-bench throughput for the same upstream llama.cpp binary set used by apps/app-llama-vulkan across four environments on the same host hardware (AMD EPYC 7543P, NVIDIA RTX 4000 Ada). The Vulkan/Venus stack runs end-to-end and emits real token-rate samples; the Unikraft-internal hosting of the same binary is tracked separately under llm.bench.vk. make llama-vulkan-bench regenerates this table.

8.7.3 Per-App Multi-Environment Evidence Matrix

Table 13 extends the comparison to all VOGUE applications (kmscube, glmark2, vkmark, app-vulkan-smoke, app-llama, app-llama-bench) across three Unikraft-specific environments (native substrate, QEMU+Unikraft CPU, QEMU+Unikraft VirtIO-GPU Vulkan) plus host baselines. All rows preserve explicit claim boundaries, including blocked-documented GPU acceleration rows. Each row records a claim_allowed and claim_forbidden boundary, matching

Table 12: LLAMA-VK throughput across same-host environments using the upstream llama-bench binary built with -DGGML_VULKAN=1, -DGGML_CUDA=1, and a plain CPU build. Model: tiny-random-LlamaForCausalLM-Q2_K.gguf (4.12 M parameters, 4.17 MiB). Vulkan/GPU vs CPU/1T: pp512 $\times$ 10.35, tg128 $\times$ 3.32. Vulkan/GPU vs CUDA/GPU tg128 ratio = 0.54 (CUDA leads on NVIDIA hardware, as expected). The same upstream binary is targeted by apps/app-llama-vulkan.

Env	Label	Backend	GPU	pp512 t/s	tg128 t/s
<i>cpu-1t</i>	Host CPU (1 thread)	ggml-cpu	-	20 590.36	725.11
<i>cpu-32t</i>	Host CPU (32 threads)	ggml-cpu	-	66 328.38	3 047.15
<i>vulkan-gpu</i>	Host Vulkan	ggml-vulkan	NVIDIA RTX 4000 Ada Generation	213 050.36	2 409.60
<i>cuda-gpu</i>	Host CUDA	ggml-cuda	NVIDIA RTX 4000 Ada Generation	232 752.06	4 472.76

Table 13: Per-app multi-environment evidence matrix. Blocked rows are documented expected states. Native substrate rows use the fake VirtIO-GPU backend. QEMU+Unikraft rows do not promote xport.qemu-vgpu unless the current generated matrix passes it; ring protocol evidence remains native unless same-run QEMU proof exists (proto.venus-ring=PASS natively).

App	Environment	Status	Evidence / Note
<i>kmscube</i>	Native fake backend	pass	gfx.kmscube.sw: 60 frames @ 469.3 FPS (640x480, BG)
<i>kmscube</i>	QEMU + Unikraft CPU (2D path)	pass-substrate:cpu	gfx.kmscube.sw path is substrate evidence; xport.q
<i>kmscube</i>	QEMU + Unikraft VirtIO-GPU Vulkan	pass	K1: same-run QEMU frame proof passes.
<i>glmark2</i>	Native fake backend	pass	gfx.glmark2.sw: 120 frames @ 187.8 FPS (1280x800)
<i>glmark2</i>	QEMU + Unikraft CPU (2D path)	pass-substrate:cpu	gfx.glmark2.sw path: substrate FPS measured native
<i>glmark2</i>	QEMU + Unikraft VirtIO-GPU Vulkan	blocked:not-planned	No accelerated glmark2 scene gate is currently imp
<i>vkmark</i>	Native substrate (vk.drm-shim+vk.iced I	pass	gfx.vkmark: ICD init, Venus context, 10 scenes; FP
<i>vkmark</i>	Baremetal Vulkan GPU (NVIDIA)	pass	Host NVIDIA baseline: clear=85000 fps.
<i>vkmark</i>	Baremetal Vulkan CPU (llvmpipe)	pass	Host llvmpipe baseline: clear=3200 fps.
<i>vkmark</i>	QEMU + Unikraft VirtIO-GPU Vulkan	blocked:render-payload-not-implemented	QEMU vkmark scene rendering is not implemented yet
<i>vulkan-smoke</i>	Native substrate (Venus capset probe)	pass	vk.smoke: host Vulkan alloc/map/fence baseline + V
<i>vulkan-smoke</i>	QEMU + Unikraft VirtIO-GPU Vulkan	blocked:render-payload-not-implemented	QEMU smoke rendering is not implemented yet.
<i>llama-upstream</i>	QEMU + Unikraft CPU bench-only	pass	True llama.cpp CPU appliance: main() selects bench
<i>llama-upstream</i>	QEMU + Unikraft CPU server-only	pass	CPU server appliance reaches READY directly from m
<i>llama-upstream-vk</i>	Static ggml-vulkan/Venus dispatch	pass	Minimal local support layer for upstream ggml-vulk
<i>llama-upstream-vk</i>	QEMU + Unikraft Vulkan bench-only	pass	True llama.cpp Vulkan appliance with same-run PASS
<i>llama-upstream-vk</i>	QEMU + Unikraft Vulkan server-only	pass	Vulkan server appliance reaches model-loaded READY

VOGUE’s evidence-gate discipline. make app-multi-env-bench generates this table from the current artifact state.

8.8 RQ5: Current Real-Driver and Venus Stage

The Venus/Vulkan substrate evidence has advanced to run-time PASS on the evaluation host. make venus-check records the QEMU probe artifacts and make stage-check verifies the real control-queue surface and production Kraft/config/build artifacts. The former modern-PCI discovery blocker is resolved for this host. The guest-side Venus bootstrap command-serialization path (libukvenus) provides Mesa-compatible VkCommandTypeEXT framing (venus_cs.c) for the

initial Vulkan bootstrap commands, capset query, context creation, and SUBMIT_3D transport (venus_init.c).

A wire-format correctness audit against Mesa’s vn_protocol_driver_instance.h and virglrenderer’s vn_protocol_renderer_instance.h revealed two encoding bugs that would prevent virglrenderer from parsing the serialized commands. Both are now fixed: (1) uk_venus_encode_uint64 previously inserted 4 bytes of alignment padding before each 64-bit field; Venus is a PACKED protocol with no inter-field alignment, so the padding was removed; (2) array presence fields (ppEnabledLayerNames, ppEnabledExtensionNames,

Table 14: Generated Venus/readiness gates. Blocked rows are explicit non-claims for acceleration.

Row	Gate	Status	Allowed conclusion
<i>VSTAT</i>	real-driver static gate	pass	controlq implementation present; not GPU performance
<i>xport.gemuvgpu</i>	QEMU Venus probe	pass	acceleration claim only if pass
<i>vk.readiness</i>	Venus perf artifact	ready-for-venus-smoke	blocked rows are readiness evidence only

Table 15: Generated current-stage completeness gate. The gate checks design alignment, generated tests, benchmarks, paper linkage, and blocked acceleration non-claims.

Gate	Status	Reviewer interpretation
<i>Alignment</i>	pass	Unikraft design rules and non-reimplementation boundaries
<i>Stage</i>	pass	Real controlq driver present; QEMU blocker named
<i>Benchmarks</i>	pass	8 generated benchmark/readiness rows
<i>Real path</i>	pass	production Kraft/config/build evidence selects real backend
<i>Evaluation</i>	pass	27 evidence rows
<i>Governance</i>	pass	developer/release split plus external manifest ownership
<i>STK</i>	out-of-scope	STK porting dropped per plan.md §0.5

`pQueueCreateInfo`, `pQueuePriorities`, `pPhysicalDevices`) were encoded as `uint64` pointer flags, but Mesa’s decoder reads them as `uint32` `array_size` values via `vn_encode_array_size()` — fixed in `vkCreateInstance`, `vkCreateDevice`, and `vkEnumeratePhysicalDevices`. The native test suite confirms the corrected layout: `venus_cs_test` checks packed encoding and correct field offsets for all bootstrap commands.

A registry checker validates the command constants against local Vulkan-Docs `vk.xml`, Mesa generated Venus protocol headers, and `VK_EXT_command_serialization.xml`. The `proto.venus-enc` row is PASS.

The Mesa-compatible Venus ring-buffer protocol is now implemented in `libukvenus` (`proto.venus-ring` row: PASS). `uk_venus_ring_register()` allocates a `HOST3D_GUEST` blob, zero-initializes the shared-memory control fields (head at offset 0, tail at offset 64, status at offset 128), computes `buf_size` as the largest power-of-2 fitting in the remaining blob space, and submits `vkCreateRingMESA` (command type 188) via `SUBMIT_3D`. `uk_venus_ring_cmd_write()` copies commands into the circular buffer with wrap-around; `uk_venus_ring_cmd_flush()` stores the guest tail via a volatile write and sends `vkNotifyRingMESA` (command type 190); `uk_venus_ring_cmd_wait()` spin-polls the host-written head field until commands are consumed. The native test suite verifies 24 ring-protocol invariants (`ring_proto`) and a throughput probe (`ring_perf`); all pass. The encoding of `vkCreateRingMESA` follows Mesa’s `vn_protocol_driver_transport.h` exactly: `ring_id` (`uint64`), `pCreateInfo` pointer flag (`uint64=1`), `sType=VK_STRUCTURE_TYPE_RING_CREATE_INFO_MESA`

(1000384000), `pNext=NULL`, `flags=0`, `resourceId`, `offset=0`, `size`, `idleTimeout=0`, and the five layout offsets plus `extraOffset/extraSize`.

`gfx.kmscube.submit` and `gfx.kmscube.frame` pass on the evaluation host with same-run `virgl` submission and pixel proof. On other hosts, reviewer-facing PASS claims still require a fresh `make kmscube-check` artifact.

The reviewer-facing current-stage gate combines the design-alignment, real-path, stage-audit, benchmark-summary, evaluation-matrix, paper, and documentation checks into one generated report. Table 15 is not a new performance result; it is a completeness check that the artifact, paper, and claim taxonomy agree before any accelerated rows are promoted.

8.9 RQ4: Footprint and Startup

Table 16 compares the measured VOGUE unikernel images with a minimal Linux kernel reference. The comparison follows the Unikraft paper’s methodology: report concrete image sizes and make the baseline explicit. We do not claim that image size alone proves performance; it quantifies the deployment cost of carrying the graphics substrate.

8.10 Claim Boundaries

The main risk in this project is overclaiming. VOGUE enforces explicit interpretation rules. The complete generated evidence matrix has **27 rows: 27 PASS and 0 blocked** on the evaluation host. Each rule below states what a PASS row allows and what it does not allow.

Table 16: Image-size comparison.

Configuration	Size	Notes
<i>VOGUE 2D image</i>	292 KB	Graphics-substrate image.
<i>VOGUE virgl probe</i>	336 KB	GL-capable path.
<i>Linux kernel</i>	9.2 MB	No initramfs.
<i>Linux + initramfs</i>	10.5 MB	Reference VM.

Table 17: Startup comparison.

Configuration	Boot	Scope
<i>VOGUE 2D</i>	10–11 ms	entry to app-ready, 5 runs.
<i>VOGUE virgl probe</i>	251–252 ms	host virgl init included.
<i>Linux + initramfs</i>	857–861 ms	entry to init-ready.

- `gfx.kmscube.sw` is not K1: software-rendered frames prove application compatibility and display output, not virgl acceleration.
- A successful capset probe or `proto.real-driver` pass proves protocol readiness, not rendering.
- `xport.qemu-vgpu` is PASS on the evaluation host; on other hosts, transport PASS still requires a complete current QEMU GL/Venus probe artifact.
- `proto.venus-enc` is PASS: the Venus encoder uses PACKED wire format and `uint32 array_size` per Mesa’s `vn_encode_array_size()`; command IDs validated against Vulkan-Docs `vk.xml` and Mesa headers.
- `proto.venus-ring` is PASS for the native ring protocol: `libukvenus` implements the Mesa ring protocol and 24 `ring_proto` native checks pass. QEMU screendump/frame evidence is not promoted unless the same current appliance run also passes the relevant runtime row.
- `gfx.kmscube.submit` is PASS on the evaluation host; native virgl encoder tests alone are not sufficient on other hosts.
- `gfx.kmscube.frame` is PASS on the evaluation host; pixel-correct rendering requires the same K1 run plus pixel colour-band proof, not stale artifacts.
- The `glmark2` row proves the official-source scene-clear adapter, not the full asset-heavy benchmark suite.
- `vk.drm-shim` proves the Mesa/Linux `virtgpu` DRM `ioctl` surface is correctly translated to `VirtIO-GPU` protocol; it does not claim Vulkan rendering inside Unikraft.
- `gfx.vkmark` is PASS for the substrate: `vk.icd` (`libukvk_icd`) initializes the Vulkan ICD over `vk.drm-shim` and enumerates 10 benchmark scenes. Scene fps scores require same-run Venus rendering evidence.
- `host.baseline.vk` is PASS as a host Linux/QEMU/Venus baseline only. `host.vk.probe`, `host.bench.vk.run`, and `host.bench.vk` require same-run Unikraft runtime JSON and cannot be satisfied by host-baseline JSON alone.
- The multi-environment comparison (Table 11) is interpreted through the claim matrix, not as a blanket PASS table. ENV11 (`vk.ggml-dispatch` static `dispatch`) passes 164/164 checks and has no `pp512/tg128` because it is a dis-

patch-correctness gate. The authoritative Unikraft Vulkan claims come from same-run runtime artifacts: ENV10 is PASS on the evaluation host, while ENV9 remains blocked pending a `llvmpipe`-specific same-run artifact.

- `bld.uk.vk` is PASS: `vogue-llama-upstream-vk_qemu-x86_64` links `ggml-vulkan` and Venus libraries under the Unikraft clang toolchain. Build-pass confirms toolchain compatibility; runtime claims still require the runtime rows.
- `llm.bench.cpu` is PASS: upstream `llama.cpp` CPU path runs inside Unikraft unikernel via 9pfs model delivery with sources unmodified. The current evidence reports `pp512=9.1`, `tg128=7.7`; GPU acceleration is not claimed by this CPU row.
- `llm.bench.vk` and `llm.bench.vk.real` are PASS on the evaluation host with real Venus artifacts. The original low-throughput failure mode was overly granular Venus submission during decode; enabling batched submission and explicit `llama.cpp` batch sizing raises the latest same-run artifact to `pp512=2232.1`, `tg128=160.2`, with three same-host post-change runs reporting `tg128` median 139.9. Remaining work is request-level server throughput and model-load latency, not basic decode-path rescue.

These rules mirror the discipline in the reference papers: results are useful only when the baseline, workload, and allowed conclusion are explicit.

8.11 Next-Stage Roadmap (N1 / N2 / `vk.ggml-dispatch`)

The current 27-row matrix (27 PASS, 0 blocked on the evaluation host) establishes the runtime path; the next work is to keep regenerated summaries aligned, add the missing ENV9 `llvmpipe` artifact path, preserve the current-stage real-path hygiene gate, add real request-level server throughput evidence, and reduce the remaining model-load cost.

N1 – Full upstream `ggml` CPU and Vulkan sources inside Unikraft (PASS). `apps/app-llama-upstream/` provides the CPU port and `apps/app-llama-upstream-vk/` provides the Vulkan/Venus port, both with upstream sources unmodified and single-purpose Kraftfiles. The current optimization target is Vulkan token generation and server request throughput, not basic runtime enablement.

N2 — Dual model-delivery transport. Config.uk for app-llama-cpu provides a Kconfig choice between initramfs (cpio embedded in image, zero host configuration) and 9pfs (host directory served via virtio-9p, unlimited model size). make llama-cpu-run-initramfs and make llama-cpu-run-9pfs exercise both transports; the acceptance gate requires pp512/tg128 to agree within 5% across transports. Effort: 3–5 days.

vk.ggml-dispatch — Static Vulkan ICD dispatch layer inside Unikraft (PASS). libs/libukggml_vulkan/uk_vulkan_dispatch.c provides 80+ Vulkan C ABI stubs backed by Venus SUBMIT_3D encoder calls; uk_ggml_vulkan_dispatch_init() initialises over the vk.drm-shim DRM virtgpu substrate with no dlopen and no host libvulkan.so. uk_ggml_vk_loader.cpp wires VULKAN_HPP_DEFAULT_DISPATCH_LOADER_DYNAMIC_STORAGE and calls VULKAN_HPP_DEFAULT_DISPATCHER.init(uk_vkGetInstanceProcAddress), so ggml-vulkan.cpp can call Vulkan entry points without a dynamic loader. Evidence: vk.ggml-dispatch row in Table 6, ENV11 in Table 11 (164/164 checks, make llama-ggml-vk-dispatch). All stub structs are read at Vulkan 1.3 spec byte offsets, verified against the Khronos Vulkan-Docs (vk.xml v1.3.352) and the Mesa vn_protocol_driver_defines.h command-type table. Remaining work: Venus ring-buffer reads (uk_venus_ring_wait_reply) for real device property queries, virtio_gpu_resource_flush for host VRAM write coherency, and a GGUF model transport (9pfs / initramfs) to enable real inference and remove the GFLOP/s extrapolation.

9 Discussion and Limitations

9.1 Main Insight

The central lesson is that graphics support for a unikernel is not an all-or-nothing Linux-porting problem. A useful substrate can be decomposed into four separable planes: (1) a device plane that speaks VirtIO-GPU correctly, (2) an API plane that preserves source compatibility for selected applications, (3) a rendering plane that can begin with software pixels and later switch to virgl or Venus, and (4) a transport-readiness plane that tests real 3D/blob commands without pretending that they are application acceleration. This decomposition is what lets VOGUE produce meaningful evidence before a full GPU renderer exists.

9.2 What the Current Paper Proves

VOGUE proves that a small Unikraft library stack can present frames through VirtIO-GPU, support real graphics application source at the compatibility boundary, and implement the real VirtIO-GPU 3D/blob control-queue surface without importing Linux DRM or Mesa. It also proves that the project can distinguish among working display output, protocol readiness, transport blockers, and hardware acceleration. This distinction is not merely defensive writing; it is an engineering mechanism. By forcing every row to name its allowed and forbidden claim, the evaluator guides implementation priorities.

9.3 What This Does Not Yet Support

K1 virgl SUBMIT_3D delivery and pixel-correct frame evidence now have current PASS claims on the evaluation host. A K1 submit claim still requires the current K1 row itself to pass with renderer=virgl and submits_3d>0; a frame claim additionally requires same-run pixel proof with colour_band_source="pixel" and a matched colour index. Log-only SUBMIT_3D diagnostics and stale frame proofs remain blockers on hosts without the same-run artifact.

Full accelerated benchmark coverage is missing. The current glmark2 row is a scene-clear substrate proof, not a full GL benchmark. A SOSP-strength version should report at least one real visual workload with frame hashes or screenshots, one synthetic microbenchmark for transfer and fence overhead, and one Linux/QEMU baseline under the same host conditions.

Security and isolation analysis is incomplete. The current work reduces guest code size, but it does not yet quantify attack surface reduction, fuzz the command parser, or evaluate malicious-device behavior.

9.4 Why This Is Still Valuable

The staged evidence ladder — disp.2d → gfx.kmscube.sw → gfx.glmark2.sw → xport.gl-probe → K1 — is a reusable method for unikernel device work. It avoids the common systems-paper failure mode of presenting an ambitious final architecture without enough intermediate evidence to tell which component actually works. The method also clarifies what should be done next: keep the same-run KMSCube proof reproducible, broaden to accelerated visual workloads, and optimize the now-running Venus/Vulkan compute path.

9.5 Performance Implications

The software-render path performs two expensive operations per frame: CPU rasterization and a framebuffer copy into DMA memory. At 1280×800 BGRA, the copy touches roughly 4 MB per frame; at 60 FPS this is about 240 MB/s before virtqueue overhead. This is plausible for simple appliances but not a substitute for GPU rendering. Virgl/Venus should remove the full-frame CPU copy for accelerated workloads by moving geometry, command buffers, or Vulkan resources to the host renderer, but they will introduce command-encoding, serialization, host-visible memory, and renderer-latency overheads. The right future evaluation is therefore not “VOGUE faster than Linux” in the abstract, but a breakdown of boot, memory, frame latency, and CPU utilization across: VOGUE software render, VOGUE virgl, Linux DRM/Mesa, and a minimal Linux appliance.

9.6 Threats to Validity

External validity. The kmscube, scene-clear, and null-renderer workloads are intentionally small. They show feasibility, not coverage of complex GUI or game workloads.

Measurement validity. Current footprint and boot comparisons are useful, but a submission-quality evaluation

Table 18: Reviewer-driven remaining work before full SOSP-level graphics acceleration.

Pri.	Missing work	Evidence required / status
P0	Broader accelerated app benchmark	K1 same-run submit + pixel-frame proof passes on the eval host; broaden to vkmark/full scenes.
P0	Linux baseline	matched kmscube/glmak2 runs; not collected.
P0	QEMU accelerated app bench	boot, first-frame, steady frame, CPU/GPU; transport passes on eval host, broader app metrics still missing.
P1	API coverage	real/stateful/safe-stub table; partial.
P1	visual artifacts	screenshots or frame hashes; native CRC only.
P2	robustness tests	malformed resource/fence/capset cases; future.

should pin CPU frequency, repeat runs, report variance, and publish raw logs.

Implementation validity. The fake backend is excellent for command-order tests and the static gate is useful for ABI coverage, but neither substitutes for QEMU/virgl/Venus behavior. All claims that depend on host-device behavior must be backed by same-run QEMU logs and frame evidence.

9.7 Remaining Work for SOSP-Level Competitiveness
Table 18 summarizes the reviewer-driven work that cannot honestly be claimed by the current artifact. VOGUE now fixes the paper-structure, figure, and native application-performance weaknesses, but the table keeps the larger implementation gaps visible.

9.8 Root Cause Analysis of Blocked Rows

9.8.1 K1: current PASS and remaining scope

K1 is a current PASS claim on the evaluation host. The root causes that previously blocked it, and the guardrails that remain, are:

1. **Native encoder readiness is separate from QEMU runtime proof.** `virgl_encoder.c` in `libkvmvirtio_gpu` encodes `CREATE_OBJECT/SURFACE`, `SET_FRAMEBUFFER_STATE`, and `CLEAR` command streams, and the native `virgl_encoder_test` passes. This proves command construction under the fake backend, not host `virglrenderer` acceptance.
2. **Current run log must carry the required PASS marker.** `kmscube_vgpu_gl_eval.py` removes stale `frame-proof.json` data when it is not present in the current `run.log`. Submit PASS requires the current K1 row to carry `renderer=virgl` and `submits_3d>0`.
3. **Pixel proof is stricter than submit proof.** `frame_pixel_proof.json` may record `SUBMIT_3D` diagnostics, but `gfx.kmscube.frame` only passes when the screendump itself matches an encoded `CLEAR` colour band and the proof hashes the same `run.log`.

The reproduction plan is to rebuild the KMSCube appliance with KraftKit, run it under QEMU’s `virtio-gpu-gl/Venus-capable` backend as documented by QEMU [25], capture a same-run serial PASS marker, capture the QMP screendump, and rerun `make kmscube-check && make eval-check`. Without that run on a new host, native `virgl` encoder results remain substrate evidence only.

9.8.2 llama.cpp full inference: current PASS and remaining scope

Full upstream `llama.cpp` CPU and Vulkan inference now pass inside single-purpose Unikraft appliances on the evaluation host. The earlier blockers were:

1. **C++ runtime missing.** The real `llama.cpp` and `ggml` source files are C++17. Unikraft’s default build uses `nolibc`; adding a C++ runtime requires pulling `libc-musl` (for a full POSIX `libc`) plus `libcxx/libcxxabi` (LLVM `libc++`) from the Unikraft external library catalog. Without these, `std::vector`, `std::string`, and exception-handling infrastructure are absent.
2. **POSIX threads missing.** `ggml`’s CPU backend uses a thread pool (`ggml_threadpool_t`) backed by `pthread_create/pthread_join`. Unikraft’s core scheduler provides cooperative fibers (`uksched`), but the POSIX thread API (`pthread_*`) is not automatically available; it requires enabling `lib-pthread-embedded` (catalog: `libs/pthread-embedded`) or Unikraft’s `posix_thread` library.
3. **Model file unavailable.** Running inference requires loading a GGUF model weight file. Inside a Unikraft VM the filesystem is absent by default; options are (a) 9pfs host-share via `lib-9pfs + posix-vfs`, (b) ramfs with the model embedded in the `initramfs` image, or (c) a `virtio-blk` block device.

GPU acceleration is delegated to upstream `ggml-vulkan` built with `-DGGML_USE_VULKAN=1` and dispatched through Mesa Venus. VOGUE does not implement or claim a custom guest-side compute-remoting ABI. The required static Vulkan API surface is routed through `libkvmvirtio_gpu/libkvmvirtio_gpu`, covered by the 164-check `vk.ggml-dispatch` test, and now backed by same-run QEMU/Venus `llama.cpp` runtime evidence. The remaining gap is performance: prefill is strong, while token generation and HTTP serving need optimization and new request-level gates.

9.9 Full llama.cpp/ggml Unikraft Porting Plan (N1 / N2 / vk.ggml-dispatch)

The current substrate (`legacy-llama-substrate + legacy-llama-substrate`) proves every pre-condition for full inference. Advancing to real token-generation inside Unikraft follows three blocks.

N1 – Full upstream ggml CPU sources compiled for Unikraft (2–3 weeks):

apps/app-llama-cpu/ (scaffold on branch) provides the port skeleton: Config.uk selects the full C++ catalog stack (LIBMUSL, LIBCXX, LIBCXXABI, LIBUNWIND, LIBCOMPILER_RT, LIBPTHREAD_EMBEDDED) and inherits the proven ABI from legacy-llama-substrate. Makefile.uk follows the APG addlib_s pattern, pinning the same upstream llama.cpp commit used for host baselines. Upstream sources are added one-by-one per the N1.D iterative patch loop; expected patch families cover mmap flag masks (MAP_POPULATE/MAP_HUGETLB), /proc/cpuinfo thread detection → CONFIG_UKPLAT_LCPU_MAXCOUNT, dlopen backend registry → compile-time register, and <chrono> steady_clock → ukplat_monotonic_clock(). scripts/llama_cpu_runtime_eval.py captures pp512/tg128 from the guest serial log and writes results/llama/llama_cpu_run_latest.json; legacy-llama-substrate promotes from “libukllama scalar substrate” to “upstream ggml real model inference.”

N2 – Dual model-delivery transport (3–5 days):

Config.uk for app-llama-cpu already provides a Kconfig choice between initramfs (cpio embedded in image) and 9pfs (host directory via virtio-9p). Both transports mount /models/ at the same guest path; application code is identical. make llama-cpu-run-initramfs and make llama-cpu-run-9pfs exercise both; the acceptance gate requires pp512/tg128 to agree within 5%.

vk.ggml-dispatch – Full ggml-vulkan.cpp linked inside Unikraft (1–3 months):

apps/app-llama-vk/ extends N1’s source set with ggml/src/ggml-vulkan/ggml-vulkan.cpp and the embedded SPIR-V table (C.1 path). libukvk_icd is extended from ICD bootstrap to the full Vulkan compute-dispatch surface (Groups 1–7: device+queue, memory, buffers, descriptors, pipelines, command encoding, submission+sync); the libukvenus encoders already proven by legacy-llama-substrate cover the wire side. Patch families cover dlopen removal (static ICD), validation-layer bypass, and host-coherent flush. Once vk.ggml-dispatch lands, legacy-llama-substrate promotes to run-proven with real GPU pp512/tg128; ENV10 drops the Venus-proxy qualifier.

The dependency graph is: N2 (model I/O) → N1 (CPU ggml in UK, legacy-llama-substrate) → vk.ggml-dispatch (Vulkan/Venus, legacy-llama-substrate upgrade). N2 is independent of K1; vk.ggml-dispatch shares the libukvk_icd runtime extension with G7 (vkmark) but lives on its own evidence ladder.

9.10 Future Work

The current artifact has 27 generated evidence rows: 27 PASS and 0 blocked on the evaluation host. The N1/N2/vk.ggml-dispatch milestones described above are now complete for runtime bring-up:

Completed (this stage):

- N1 CPU path: apps/app-llama-upstream with upstream-unmodified llama.cpp running inside Unikraft. llm.bench.cpu PASS: pp512=9.1 t/s, tg128=7.7 t/s.

- N2 model delivery: 9pfs host-share via VirtIO-9P (CONFIG_LIBUK9P=y, CONFIG_LIB9PFS=y, CONFIG_LIBVFSCORE_AUTOMOUNT_CI_RAMFS=y). Model mounted at /mnt/model.
- vk.ggml-dispatch static dispatch and runtime: apps/app-llama-upstream-vk with libukggml_vulkan builds, dispatch tests pass, and the Vulkan bench/server appliances have same-run Venus artifacts.
- POSIX surface served entirely by upstream lib-musl external library on top of Unikraft’s posix-mmap/time/fdio/vfs libs — no project-local POSIX shim is required.

Next stage:

1. **HTTP server evidence.** llm.server.vk now reaches READY with batch_size=2048, ubatch_size=512, and prompt caching enabled, but there is still no lwIP/netdev-backed request path. Add request serving before reporting TTFT, requests/s, or aggregate server throughput.
2. **Model-load optimization.** model-load-time-check still records use_mmap=false, huge_pages=false; the remaining 4.7-7.1 s model-load path should be reduced before claiming startup wins for larger models.
3. **Linux/QEMU baseline.** Collect matched kmscube/glmark2 runs on Linux+Mesa under the same host for a direct performance comparison.
4. **Full glmark2 scene coverage and frame hashes.** Replace the current scene-clear substrate proof with at least one real visual workload, frame hashes, and a Linux Mesa baseline.
5. **Security audit.** Quantify attack surface reduction, fuzz the command parser, and evaluate malicious-device behavior.

10 Related Work

10.1 Unikernels and Library OSes

VOGUE builds directly on the unikernel and library-OS tradition. MirageOS argued that single-purpose cloud services can be built as type-safe library OSes with small images and strong specialization [16]. Unikraft later generalized the idea with a micro-library architecture, Kconfig composition, and evidence that specialization can improve footprint and performance for server workloads [12]. Drawbridge explored library OS compatibility from the opposite direction: preserve rich Windows application compatibility by moving a large OS personality into an isolated library OS [22]. VOGUE takes the Unikraft/Mirage side of this tradeoff: it does not import a rich OS personality, but instead implements the smallest graphics personality needed by selected workloads.

The Exokernel line of work exposed low-level resources to applications for application-specific management [5]. Arakis and IX similarly split control-plane policy from data-plane I/O, allowing applications to bypass kernel mediation on the fast path [2, 21]. VOGUE is narrower: it does not redesign the whole OS I/O model, but it applies the same

principle to graphics in a unikernel. The VirtIO-GPU data path becomes a library-level fast path, while the hypervisor and device protocol remain the protection boundary.

10.2 Lightweight Virtualization

Firecracker demonstrates that microVMs can provide fast startup and strong isolation for serverless workloads [1]. VOGUE is complementary: it targets what runs inside a minimal VM when the appliance needs graphics. The key question is not whether a VM can be lightweight, but whether the guest graphics stack can be made lightweight enough to preserve unikernel advantages.

10.3 GPU Virtualization

VirtIO-GPU is the device target. The protocol is specified by OASIS [20] and implemented by QEMU [25]. VirGL translates guest Gallium-style command streams to host OpenGL [30]. Venus provides a Vulkan-over-VirtIO path, and rutabaga/gfxstream supports ChromeOS/Android graphics virtualization [3, 17]. The vhost-user-gpu mechanism externalizes the GPU backend for process isolation [24].

These systems primarily define host/device mechanisms or full guest-stack integration. VOGUE’s contribution is guest-side: a small Unikraft library stack that can speak the device protocol and host selected graphics applications without Linux DRM or Mesa. K1 remains future work because host support alone is insufficient; the guest must still produce valid virgl command streams.

10.4 Minimal Graphics Stacks

Linux DRM/KMS [14], Mesa [18], GBM, EGL, and GLES form the commodity path for direct-rendered applications. VOGUE replaces this stack with a compatibility boundary, not a complete reimplementation. Headless software renderers such as llvmpipe or OSMesa provide software pixels through Mesa; VOGUE’s rasterizer is intentionally smaller and less general. The tradeoff is explicit: VOGUE gains auditability and small footprint, but loses shader and compiler coverage until virgl is implemented.

10.5 Application Porting to Unikernels

Prior Unikraft application work focuses on network and storage services. VOGUE extends the porting pattern to graphics by factoring the port into reusable shims rather than application-specific rewrites. The closest analogy is POSIX compatibility work: a narrow API surface enables unmodified source to compile, but every stub must be accounted for. VOGUE makes that accounting visible through claim rows and forbidden claims.

11 Conclusion

This paper demonstrates that a unikernel can support practical graphics display workloads without importing a full Linux graphics subsystem. VOGUE implements a standards-based VirtIO-GPU 2D pipeline, a minimal EGL/GLES2/GBM/DRM shim, and a CPU software rasterizer as Unikraft micro-libraries. It also implements the real VirtIO-GPU 3D/blob control-queue surface needed for the next acceler-

ated stage while preserving Unikraft’s design principle of reusing existing PCI, virtio, allocator, and POSIX/mmap support instead of reimplementing them.

Our evaluation validates progressive milestones: (disp.2d) a correct 2D display pipeline verified by native evidence-gated tests; (gfx.kmscube.sw) upstream *kmscube* source compiled and running against the shim with software rendering; (gfx.glmark2.sw) an additional application framework demonstrating substrate generality; (proto.real-driver, vk.readiness) real-driver ABI/readiness and benchmark-design gates; (vk.drm-shim, gfx.vkmark, vk.ggml-dispatch) Venus/Vulkan and ggml substrate gates; and, on the evaluation host, real QEMU/Venus transport, KMSCube SUBMIT_3D/pixel proof, and upstream llama.cpp Vulkan runtime with post-fix same-run pp512=2232.1 and tg128=160.2 over Venus. The remaining gaps are HTTP serving, broader vkmark scene rendering, the llvmpipe-specific ENV9 artifact, and model-load optimization rather than decode-path rescue.

The accelerated rendering path remains the most significant remaining milestone. The infrastructure for it — context management, capability queries, blob resources, UUID/map/unmap, 3D submission, Venus bootstrap command serialization with verified wire-format correctness (proto.venus-enc: PASS; PACKED encoding, uint32 array_size per Mesa protocol), and the full Mesa ring-buffer protocol (proto.venus-ring: PASS for native checks) — is in place as substrate evidence. The next concrete step is to turn this substrate into same-run QEMU/Venus transport, K1 frame, and ggml/Venus compute-offload evidence without overclaiming rows that still lack fresh runtime proof. We believe the progressive validation approach and evidence-gated evaluation framework described in this paper provide a sound methodology for reaching that milestone without overclaiming along the way.

References

- [1] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight Virtualization for Serverless Applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020. 419–434.
- [2] Adam Belay, George Prekas, Ana Klimovic, Samuel Grossman, Christos Kozyrakis, and Edouard Bugnion. 2014. IX: A Protected Dataplane Operating System for High Throughput and Low Latency. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2014. 49–65.
- [3] ChromiumOS contributors. 2024. Rutabaga: gfxstream graphics abstraction layer. Retrieved from <https://crosvm.dev/book/devices/gpu.html>
- [4] Rob Clark and Arvin Schnell. 2023. kmscube: DRM/KMS cube demo. Retrieved from <https://gitlab.freedesktop.org/mesa/kmscube>
- [5] Dawson R. Engler, M. Frans Kaashoek, and James O’Toole. 1995. Exokernel: An Operating System Architecture for Application-Level Resource Management. In *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles (SOSP)*, 1995. 251–266. <https://doi.org/10.1145/224056.224076>
- [6] ggml-org. 2024. llama.cpp: LLM Inference in C/C++.
- [7] ggml-org. 2024. GGML VirtGPU Backend.
- [8] glmark2 contributors. 2024. glmark2: OpenGL (ES) 2.0 benchmark suite. Retrieved from <https://github.com/glmark2/glmark2>
- [9] The Khronos Group. 2024. OpenGL: The Industry’s Foundation for High Performance Graphics.
- [10] The Khronos Group. 2024. Vulkan: Cross-Platform 3D Graphics and Compute.
- [11] The Khronos Group. 2026. Vulkan Guide: Loader. Retrieved from <https://docs.vulkan.org/guide/latest/loader.html>
- [12] Simon Kuenzer, Vlad-Andrei Bădoiu, Hugo Lefeuvre, Sharan Santhanam, Alexander Jung, Gauthier Gain, Cyril Soldani, Costin Lupu, Ștefan Teodorescu, Costi Răducanu, Cristian Banu, Laurent Mathy, Răzvan Deaconescu, Costin Raiciu, and Felipe Huici. 2021. Unikraft: Fast, Specialized Unikernels the Easy Way. In *Proceedings of the 16th European Conference on Computer Systems (EuroSys)*, 2021. <https://doi.org/10.1145/3447786.3456248>
- [13] Joost Lim and others. 2022. Cricket: CUDA Virtualization via API Remoting.
- [14] Linux kernel contributors. 2024. Direct Rendering Manager (DRM/KMS) subsystem. Retrieved from <https://www.kernel.org/doc/html/latest/gpu/>
- [15] Linux Kernel Documentation. 2024. VFIO – Virtual Function I/O.
- [16] Anil Madhavapeddy, Richard Mortier, Charalampos Rotsos, David Scott, Balraj Singh, Thomas Gazagnaire, Steven Smith, Steven Hand, and Jon Crowcroft. 2013. Unikernels: Library Operating Systems for the Cloud. In *Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2013. 461–472. <https://doi.org/10.1145/2451116.2451167>
- [17] Mesa contributors. 2024. Venus: Vulkan over VirtIO. Retrieved from <https://docs.mesa3d.org/drivers/venus.html>
- [18] Mesa contributors. 2024. Mesa 3D graphics library. Retrieved from <https://mesa3d.org/>
- [19] NVIDIA Developer. 2023. Confidential Computing on NVIDIA H100 GPUs for Secure and Trustworthy AI.
- [20] OASIS Technical Committee. 2023. *Virtual I/O Device (VIRTIO) Version 1.3*. Retrieved from <https://docs.oasis-open.org/virtio/virtio/v1.3/virtio-v1.3.html>
- [21] Simon Peter, Jialin Li, Irene Zhang, Dan R. K. Ports, Doug Woos, Arvind Krishnamurthy, Thomas Anderson, and Timothy Roscoe. 2014. Arrakis: The Operating System is the Control Plane. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2014. 1–16.
- [22] Donald E. Porter, Silas Boyd-Wickizer, Jon Howell, Reuben Olinsky, and Galen C. Hunt. 2011. Rethinking the Library OS from the Top Down. In *Proceedings of the 16th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2011. 291–304. <https://doi.org/10.1145/1950365.1950399>
- [23] QEMU Documentation. 2024. AMD Secure Encrypted Virtualization (SEV).
- [24] QEMU Project. 2024. vhost-user-gpu: External VirtIO-GPU backend. Retrieved from <https://www.qemu.org/docs/master/interop/vhost-user-gpu.html>
- [25] QEMU Project. 2026. virtio-gpu – QEMU Documentation. Retrieved from <https://www.qemu.org/docs/master/system/devices/virtio-gpu.html>
- [26] Typst Project. 2026. Typst PDF Export Documentation. Retrieved from <https://typst.app/docs/reference/pdf/>
- [27] Unikraft Project. 2026. kraft build – Configure and build Unikraft unikernels. Retrieved from <https://unikraft.org/docs/cli/reference/kraft/build>
- [28] Unikraft Project. 2026. Kraftfile Reference. Retrieved from <https://unikraft.org/docs/cli/reference/kraftfile/latest>
- [29] vAccel Project. 2024. vAccel: A Hardware-Agnostic Acceleration API.
- [30] virglrenderer contributors. 2024. virglrenderer: A virtual 3D GPU library. Retrieved from <https://gitlab.freedesktop.org/virgl/virglrenderer>

Table 19: Artifact claim map from the current generated evidence matrix: 27 evidence rows, 27 PASS and 0 blocked on the evaluation host. New hosts must still produce same-run artifacts before promoting QEMU/Venus acceleration or upstream Vulkan runtime rows.

Paper Claim / Evidence Row	Artifact Component	Validation
<i>disp.2d: 2D pipeline</i>	native 2D render test	native-tests PASS log
<i>proto.api-contract: API conformance</i>	VirtIO-GPU full API test	native-tests PASS log
<i>Real-driver wire ABI</i>	make -C tests proto-abi	native ABI PASS log against virtio_gpu_proto.h
<i>gfx.kmscube.sw: kmscube sw render</i>	app perf script + kmscube port	app-perf-check generated row
<i>gfx.glmak2.sw: glmak2 scene-clear</i>	app perf script + glmak2 port	app-perf-check generated row
<i>proto.real-driver: real-driver ABI</i>	proto-abi + static real-driver checks	make venus-check; PASS = protocol surface, not rendering
<i>xport.qemu-vgpu: QEMU Venus transport</i>	QEMU probe script	Current matrix: PASS on evaluation host; QEMU GL/Venus setup is required before transport PASS on any new host
<i>vk.readiness: benchmark design</i>	benchmark summary script	make benchmark-check; PASS = evaluation design exists
<i>proto.venus-enc: wire format</i>	venus_cs_test / venus_compute_test	100+ Venus-encoder native checks pass
<i>proto.venus-ring: ring protocol</i>	libukvenus ring tests in venus_cs_test	ring_proto + ring_perf checks pass
<i>vk.drm-shim: DRM virtgpu ioctl shim</i>	virtgpu_drm_ioctl_test	native test PASS
<i>vk.icd: Vulkan ICD bootstrap</i>	vk_icd_bootstrap_test	native test PASS
<i>legacy-llama-substrate: GGUF model view</i>	ukmodel_test	native test PASS
<i>legacy-llama-substrate: ggml substrate</i>	ggml_uk_test	native test PASS
<i>legacy-llama-substrate: llama adapter</i>	ukllama_test	native test PASS
<i>legacy-llama-substrate: staged C++ probe</i>	scripts/llama_vulkan_eval.py cmd_cpp()	Current matrix: blocked:cpp-runtime-not-proven; rerun before claiming PASS
<i>legacy-llama-substrate: unikernel Venus runtime</i>	scripts/llama_vulkan_eval.py cmd_uk()	Current matrix: blocked:vulkan-uk-not-yet-run; static/API rows remain separate PASS evidence
<i>bld.uk.vk</i>	scripts/llama_vulkan_eval.py cmd_n3_build()	vk.ggml-dispatch static-ICD image builds; n3_build_passed.json
<i>llm.bench.cpu: pp512</i>	scripts/llama_vulkan_eval.py cmd_upstream_cpu()	PASS; current CPU artifact passes; Vulkan rows remain separate
<i>llm.bench.vk</i>	scripts/llama_vulkan_eval.py cmd_upstream_vk()	BLOCKED until rerun evidence; required API covered; run-time needs EGL render node
<i>llm.bench.vk.real: upstream+real Venus</i>	derived from llm.bench.vk	BLOCKED locally: no EGL render node for QEMU Venus
<i>gfx.kmscube.submit/gfx.kmscube.frame</i>	app-kmscube virgl path + frame proof	Current matrix: PASS on evaluation host; stale frame proofs are removed and native virgl encoder tests remain separate substrate evidence

12 Artifact Appendix

12.1 Artifact Overview

The VOGUE artifact contains the Unikraft VirtIO-GPU library stack, application ports, build scripts, native test suite, evaluation scripts, paper-generation scripts, and raw result logs used to produce the evidence matrix in this paper. The artifact is deliberately evidence-gated: software-render display results, real-driver ABI/readiness checks, QEMU/Venus transport status, native substrate performance, and future accelerated-rendering claims are separate rows.

12.2 Artifact Claims

Generated performance and stage artifacts are written to the results directory and included in the paper through

generated tables. The QEMU/Kraft and QEMU/Venus gates write structured blocker files when the local external tool-chain or transport blocks a run, and those blockers never count as K1 or Venus acceleration success.

12.3 External Dependency Inventory

All external code referenced by this artifact is listed below with exact local paths and versions. This table allows a reviewer to verify reproducibility and identify what must be installed before running any gate.

12.3.1 Venus Protocol Header Inventory

The Venus protocol wire format is defined by Mesa’s `vn_protocol_driver_defines.h`. The following tokens from that header are the load-bearing pieces for VOGUE’s encoder correctness:

Table 20: External dependency inventory. Paths are referenced through the variables defined in `config/external_paths.json`; the defaults match the evaluation host and can be overridden via the corresponding environment variables. The `cmake/unikraft-clang.cmake` toolchain file consumes the same variables.

Component	Local Path	Version / SHA	Purpose
<i>Mesa Venus protocol headers</i>	<code>\$VENUS_PROTOCOL_ROOT/include/</code>	KhronosGroup/venus-protocol	Venus wire-format type definitions; packed struct layout
<i>Vulkan Headers</i>	<code>\$VULKAN_HEADERS_INCLUDE</code>	Khronos Vulkan-Headers v1.3.352	Vulkan API headers for <code>libukggml_vulkan</code> , <code>libukvk_icd</code> , <code>apps/app-llama-upstream-vk</code>
<i>SPIR-V Headers</i>	<code>\$SPIRV_HEADERS_INCLUDE</code>	KhronosGroup/SPIRV-Headers (matched to Vulkan 352)	SPIR-V encoding enums for <code>ggml-vulkan.cpp</code> shader table
<i>llama.cpp (upstream)</i>	<code>\$LLAMA_ROOT</code> (host build); in-tree at <code>apps/app-llama-upstream*/</code>	SHA <code>fcae601e4</code> (ggml-org/llama.cpp)	CPU baseline benchmarks; upstream-unmodified Unikraft port; ggml-vulkan compute
<i>libvulkan.so</i>	<code>/usr/lib/x86_64-linux-gnu/libvulkan.so.1</code>	system Vulkan loader	Host-side Vulkan tests (make <code>vulkan-tests</code>)
<i>virglrenderer</i>	host QEMU dependency	linked into QEMU 11.0	Host-side Venus capset decode and <code>SUBMIT_3D</code> dispatch
<i>QEMU</i>	<code>qemu-system-x86_64</code>	11.0 (required for Venus capset id=4)	VM execution for all appliance eval targets
<i>KraftKit / kraft</i>	<code>kraft</code> in PATH	v0.9+ recommended	Unikraft appliance build: make <code>kmscube-build</code> , make <code>llama-vulkan-n3-build</code>
<i>Unikraft tree</i>	<code>../unikraft</code> (sibling checkout)	aligned with Kraftfiles in <code>kraft/</code>	Unikraft kernel and library build system
<i>CMake toolchain</i>	<code>cmake/unikraft-clang.cmake</code>	in-tree	Forces SPIR-V + Vulkan v352 headers for clang cross-builds
<i>typst</i>	<code>typst</code> in PATH	v0.11+	Paper compilation: make paper

Table 21: Venus protocol header tokens verified in the VOGUE artifact. The `real_virtio_gpu_path_check.py --check` gate fails if any token is absent from `libs/libukvirtio_gpu/virtio_gpu_proto.h`.

Token / Field	Significance for VOGUE
<code>VN_CS_ENCODER_BUFFER_SIZE</code>	Ring command-buffer segment size; must match <code>virglrenderer</code> decoder expectations
<code>vn_encode_array_size(enc, n)</code>	Encodes array presence as <code>uint32_t n</code> ; zero means absent; VOGUE encoder replicates this protocol in <code>libukvenus</code>
<code>VN_ENCODE_* PACKED</code> layout	No inter-field alignment padding; all structs use <code>__attribute__((packed))</code> or equivalent; <code>virglrenderer</code> assumes this
<code>VIRTIO_GPU_F_VIRGL</code> , <code>VIRTIO_GPU_F_RESOURCE_BLOB</code>	Feature bits negotiated at probe; required before any 3D or blob command is issued
<code>VIRTIO_GPU_CMD_SUBMIT_3D</code> , <code>VIRTIO_GPU_RESP_OK_MAP_INFO</code>	<code>VIRTIO_GPU_CMD_RESOURCE_CREATE_BLOB</code> , Control-queue command IDs; verified by <code>real_virtio_gpu_path_check.py</code>

12.4 Hardware and Software Requirements

VOGUE’s evidence rows span three tiers with different hardware requirements. The native-test and software-render tiers require only a Linux host; the Venus/Vulkan tier additionally requires a Vulkan-capable host GPU and QEMU 11.0.

12.5 Official-Documentation-Grounded Environment Plan

The build environment follows the upstream documentation rather than project-local assumptions. KraftKit’s official `kraft` build reference defines target selection by platform/

architecture (for example `qemu/x86_64` [27], and the Kraftfile reference defines targets as platform/architecture tuples [28]. QEMU documents the relevant graphics backend as `virtio-gpu-gl` with `hostmem`, `blob=true`, and `venus=true` for Venus/Vulkan [25]; Mesa documents Venus as Vulkan command serialization over `VirtIO-GPU` [17]. Khronos documents that the Vulkan loader maps applications to layers and installable client drivers [11]. Typst documents that compile produces the paper PDF [26].

Table 22: Hardware and software requirements by tier. Source for Venus host GPU requirements: Mesa Venus documentation at <https://docs.mesa3d.org/drivers/venus.html>. Any Vulkan 1.1 GPU driver on Linux is sufficient; no specific GPU vendor or model is required.

Component	Minimum Requirement	Notes
Host CPU architecture	x86_64 (primary); arm64 untested	KVM optional for native-test tier; recommended for appliance tier
Host OS	Linux (kernel ≥ 5.4 for VirtIO-GPU 2D; kernel ≥ 5.16 for blob/Venus features)	Ubuntu 22.04 or later recommended; any distro with QEMU 11.0 available
Host GPU — native and sw-render tiers	None required	make native-tests, make app-perf-check, make eval-check, make paper run without any GPU
Host GPU — Venus/Vulkan tier	Any GPU with a Vulkan 1.1 driver on Linux	Tested drivers per Mesa Venus documentation: ANV ≥ 21.1 , RADV ≥ 21.1 , NVIDIA proprietary ≥ 570.86 , Turnip ≥ 22.0 , Lavapipe ≥ 22.1 (software). Requires VK_KHR_external_memory_fd. GPU vendor, architecture, and model are not constrained beyond Vulkan 1.1 support.
VMM	QEMU ≥ 7.0 for 2D VirtIO-GPU; QEMU 11.0 for Venus capset id=4	Must be built with virglrenderer support (<code>--enable-virglrenderer</code>). Venus requires <code>virtio-gpu-gl-pci.blob=true,venus=true</code> . The recorded probe used QEMU 11.0.
QEMU KVM flag	<code>-accel kvm,honor-guest-pat=on</code> recommended for Venus	Per Mesa Venus documentation: <code>honor-guest-pat=on</code> required for correct Venus host-visible memory behavior. KVM without this flag may cause coherency issues.
virglrenderer	Version linked into QEMU build	Must include Venus support (Vulkan/capset-4 path). No separate minimum version number specified; use the version distributed with your QEMU 11.0 build.
Guest OS	Unikraft (this repository)	Unikraft replaces Linux DRM/KMS; guest kernel ≥ 5.16 requirement applies to Linux guests only
Host Vulkan loader	libvulkan.so.1 for host-side Vulkan tests	Used by <code>make vulkan-tests</code> and <code>tests/vk_icd_bootstrap_test</code> ; not required for native-test tier
Build toolchain	clang or gcc; KraftKit + sibling <code>../unikraft</code> tree	clang/gcc for native tests; KraftKit for appliance builds; typst for paper

Table 23: Environment, problem, root-cause, and fix plan grounded in upstream documentation and current generated artifacts.

Task / problem	Observed reason	Fix / verification plan
Kraft/QEMU appliance build	Requires a KraftKit target and sibling Unikraft tree; interrupted or missing builds leave runtime rows blocked.	<code>kraft build --target qemu/x86_64</code> through <code>make kmscube-build</code> ; then <code>make kmscube-check</code> .
QEMU/Venus runtime	QEMU must provide a GL/Venus-capable VirtIO-GPU backend with <code>hostmem/blob/Venus</code> enabled; otherwise QEMU probes/runtime rows stay blocked.*.	Install/choose QEMU with virglrenderer/Venus, use a GL-capable display backend, then rerun <code>make venus-check</code> and <code>make eval-check</code> .
KMSCube submit/frame proof	Native encoder tests are not same-run host proof; stale frame artifacts must not pass.	Require current run-log PASS marker plus pixel colour-band proof; run <code>make kmscube-check && make eval-check</code> .
Vulkan/LLM runtime rows	Host baseline JSON is the wrong domain for Unikraft runtime claims.	Require row-compatible <code>evidence_id</code> , Unikraft run source, and required fields before PASS; same-run Unikraft artifacts now exist on the evaluation host.
Paper build	Typst and fonts are host tools; missing fonts are warnings if typst compile exits zero.	<code>make paper-check</code> paper; treat non-zero Typst exit as failure.
Performance noise	Native software/substrate timing varies on non-isolated hosts.	<code>make perf-check</code> ; best-of-N is the smoke-gate value, JSON persists median/all samples for analysis.

12.6 Build and Evaluation Instructions

The full local verification gate runs native tests, application

substrate benchmarks, paper compilation, claim audit, and QEMU/Kraft blocker capture.

Table 24: Native test group targets. All run on the host without QEMU, Unikraft, or a GPU. See tests/README.md for expected pass strings and evidence mapping.

Target	Test group	Evidence rows
<i>make test-core</i>	VirtIO-GPU core, DMA, shims	proto.real-driver, gfx.kmscube.sw, disp.2d
<i>make test-venus</i>	VirtIO-GPU 3D, Venus encoder, virgl	vk.drm-shim, vk.icd, proto.venus-enc, proto.venus-ring
<i>make test-llm</i>	LLM/ggml substrate	legacy-llama-substrate, legacy-llama-substrate, legacy-llama-substrate
<i>make test-n3</i>	vk.ggml-dispatch static Vulkan dispatch (164 checks)	vk.ggml-dispatch
<i>make proto-abi</i>	VirtIO wire-ABI struct/offset/feature	proto.real-driver

```
# Clone with submodules
git clone <repo> vogue && cd vogue

# Full local verification gate
make verify

# Fast developer gate (no QEMU required)
make test-fast

# Individual gates used by reviewers
make native-tests
make app-perf-check
make eval-check
make paper
make claim-check
make stage-check
make benchmark-check
make venus-check
make paper-check
make kmscube-build
make kmscube-check
```

Listing 2: Build and evaluation commands. Run “make verify” for the complete gate; use individual targets to reproduce specific evidence rows.

The native test suite is organized into five focused groups. Reviewers can reproduce individual evidence chains without running the full suite:

Expected developer test output includes all 13 host-native substrate tests plus the ABI guard:

Expected application substrate output:

If the external Kraft/Unikraft appliance build or QEMU/Venus probe fails, the target records a structured blocker rather than promoting K1. A K1 pass requires a same-run log with an accepted VirtIO-GPU device, capset/feature evidence, 3D or Venus command submission, frames ≥ 3 , and frame hashes or screenshots tied to the run.

12.7 Troubleshooting

12.8 Artifact Badge Target

We target **Artifacts Available** and **Artifacts Evaluated – Functional** for the bounded PASS claims in the current generated matrix. The current artifact has 27 evidence rows: 27 PASS and 0 blocked on the evaluation host, including xport.qemu-vgpu, gfx.kmscube.submit/gfx.kmscube.frame, host.vk.probe, host.bench.vk.run, host.bench.vk, llm.bench.vk, llm.server.vk, and llm.bench.vk.real. The artifact still requires fresh row-com-

```
make native-tests
dma_buf_test passed alignment=4096 sg=1 pool=2
virtio_gpu_full_api_test passed capsets=5 fences=5 submits_3d=1
blobs=1
virtio_gpu_2d_render_test: PASS frames=3 transfers=3 flushes=3
fences=6
kmscube_compat_test passed mode=1280x800
virtgpu_drm_ioctl_test: all checks passed
vk_icd_bootstrap_test: all checks passed
venus_cs_test: all checks passed
venus_compute_test: all checks PASS
virgl_encoder_test: all checks passed
ukmodel_test: all checks passed
ggml_uk_test: all checks passed
ukllama_test: all checks passed
ggml_vk_dispatch_test: all checks PASS [160 passed, 0 failed]
make -C tests proto-abi
virtio_gpu_proto_abi_test passed ctrl_hdr=24 display_info=408
edid=1056
```

Listing 3: Expected make native-tests output. All 13 tests plus the ABI guard pass without QEMU, a GPU, or prebuilt ggml libraries.

```
gfx.kmscube.sw: kmscube avg_frame_ms=<measured> fps=<measured>
transfers=60 flushes=60 fences=120
gfx.glmark2.sw: glmark2 scene clear avg_frame_ms=<measured> ...
```

Listing 4: Expected application substrate benchmark output. Measured values vary by host; the structure and row labels must match.

patible same-run artifacts before promoting those rows on any different host.

12.8.1 Next-Stage Reproduction Notes

The pinned upstream ggml-vulkan.cpp API surface is covered by the static dispatch table and checked by make llama-vulkan-api-coverage. The runtime blocker for llm.bench.vk, llm.bench.vk.real, and llm.bench.vk.run is host QEMU/Venus setup: QEMU virtio-gpu-gl-pci,venus=true requires virglrenderer to initialize through a GL/EGL-capable display backend. In this sandbox the run reports blocked:no-egl-render-node.

Table 25: Common issues and fixes.

Symptom	Likely Cause	Fix
<i>native tests fail</i>	missing compiler/header path	check tests/Makefile and production library includes
<i>kmscube-build records blocked build</i>	external Kraft/Unikraft compiler mismatch	inspect the build log; fix toolchain before claiming K1
<i>black QEMU display</i>	scanout or flush not issued	verify SET_SCANOUT, TRANSFER_TO_HOST_2D, and RESOURCE_FLUSH order
<i>virtio-gpu-gl init fails</i>	no host OpenGL/virglrenderer, or virglrenderer not linked into QEMU	use 2D software path or install GL-capable QEMU stack built with --enable-virglrenderer
<i>venus probe regresses to blocked:modern-pci-unsupported</i>	Unikraft libvirtio_pci patch for modern VirtIO-GPU ID 0x1050 missing or not selected	apply/reuse the modern virtio-pci support patch before claiming xport.qemu-vgpu; still require K1 frame proof before claiming Venus acceleration
<i>blocked:egl-not-initialized or blocked:no-pass-line on Venus/Vulkan targets</i>	missing image, missing KVM/Venus support, or stale evidence from old QEMU EGL launch	Use a GL-capable QEMU display backend (for example egl-headless,gl=on) on a host with a working render node; inspect structured JSON before promoting any claim. These blockers are resolved on the evaluation host.
<i>Host Vulkan 1.1 not available</i>	Host GPU driver older than listed minimums, or Vulkan loader not installed	Install a Vulkan 1.1 driver (any of: ANV ≥ 21.1, RADV ≥ 21.1, NVIDIA proprietary ≥ 570.86, Lavapipe ≥ 22.1). vulkaninfo confirms capabilities. VK_KHR_external_memory_fd must be present.

```

make llama-vulkan-api-coverage # required API surface: pass
make llama-ggml-vk-dispatch   # native static-dispatch substrate: pass
make llama-vulkan-n3-run      # pass or structured blocked:no-egl-render-node
make llama-upstream-vk-run     # pass or structured blocked evidence

```

Listing 5: Verification commands for the ggml-vulkan/Venus path. Runtime claims require same-run PASS evidence; blocked rows remain non-claims.

13 GPU Driver Ecosystem Survey

This appendix surveys the landscape of GPU virtualization protocols, host-side rendering backends, guest graphics stacks, and emerging compute-remoting approaches, placing VOGUE’s current implementation in context at each layer. The goal is to map what exists, characterize where VOGUE intersects each technology, and identify the dependency chain required to advance blocked evidence rows.

13.1 Virtualization Substrate

VirtIO and VirtIO-GPU. VirtIO [20] establishes the standard para-virtualization device model: feature negotiation at probe time, shared virtqueue descriptor rings, and asynchronous notifications. VirtIO-GPU extends this model with a dedicated GPU device class that multiplexes a control queue and a cursor queue over the same transport. VOGUE’s `libkvirtio_gpu` implements the guest side of this protocol directly against Unikraft’s VirtIO transport, avoiding any Linux kernel infrastructure. The current implementation covers the full 2D command surface: `RESOURCE_CREATE_2D`, `RESOURCE_ATTACH_BACKING`, `TRANSFER_TO_HOST_2D`, `SET_SCANOUT`, `RESOURCE_FLUSH`, and fence synchronization. This is sufficient for evidence rows `disp.2d` and `gfx.kmscube.sw`.

VirtIO-GPU 3D / VIRTIO_GPU_F_VIRGL. The 3D feature bit promotes the device from a framebuffer adapter to an opaque command-stream forwarder. When negotiated, the guest may issue `CTX_CREATE`, `GET_CAPSET_INFO`, `GET_CAPSET`, and `SUBMIT_3D` commands; the host forwards the command buffer to `virglrenderer` for OpenGL translation. VOGUE’s `libkvirtio_gpu` implements the transport commands for context lifecycle, resource attach/detach, and 3D submission, but the guest still lacks the final accelerated-frame path: `virgl` command-stream coverage for OpenGL and Venus frame-proof wiring for Vulkan. This is the K1 blocked row. The former QEMU 11.0 modern VirtIO-PCI discovery blocker for device ID `0x1050` is resolved by the Unikraft patch carried in this artifact.

Resource blobs and host-visible memory. VirtIO-GPU 1.1 introduced resource blobs: a mechanism for creating resources backed by host memory, guest memory, or shared guest-host memory without an explicit `TRANSFER_TO_HOST_2D` round-trip. Blob resources are the transport foundation for Venus (Vulkan) and for high-bandwidth ML-weight loading. VOGUE now implements the blob, UUID, map, and unmap control-queue commands in the real backend and validates their ABI/static presence. This is necessary Venus-readiness evidence, but it is not a Vulkan application run until the real appliance command stream is exercised in QEMU and produces frame/host-processing evidence.

13.2 Host-Side Rendering and Backend Isolation

QEMU `virtio-gpu` and `virtio-gpu-gl`. QEMU [25] exposes VirtIO-GPU through multiple device variants. The baseline

`virtio-gpu` device handles 2D commands and provides a software display without host GPU involvement. The `virtio-gpu-gl/Venus-capable` variants link against `virglrenderer` and route 3D or Vulkan command streams to host OpenGL/Vulkan drivers. VOGUE targets these variants in stages: the 2D software path runs today; the real 3D/blob control surface is implemented; the current generated matrix records QEMU/Venus transport as PASS on the evaluation host; the native and QEMU Venus ring-buffer protocol passes; and the remaining work is broader rendering coverage and performance.

VirGL and `virglrenderer`. VirGL [30] is a translation layer that interprets guest Gallium-style command streams and emits corresponding host OpenGL calls. It operates entirely on the host; the guest is responsible for generating syntactically and semantically correct commands. This division of responsibility is important for VOGUE: the guest command encoder and the host renderer are independent components. A bug in the guest encoder produces `virglrenderer` errors or silently wrong frames; it cannot be debugged by examining `virglrenderer` source alone.

Venus. Venus [17] provides a VirtIO-GPU Vulkan serialization protocol. Guest Vulkan API calls are serialized by the Mesa guest driver and forwarded through VirtIO-GPU command streams; `virglrenderer` deserializes and dispatches them as host Vulkan calls. Venus requires blob resources and host-visible memory for zero-copy command and buffer sharing. VOGUE now targets the Venus transport substrate — blob resources, context operations, UUIDs, map/unmap, and `SUBMIT_3D` — with verified wire-format correctness: the encoder uses PACKED layout (no inter-field alignment padding) and encodes array presence as `uint32 array_size` values per Mesa’s `vn_encode_array_size()` protocol, matching `virglrenderer`’s decoder (`vkcr_context_submit_cmd`). The current evaluation host proves the QEMU/Venus path with same-run transport, frame, and `llama.cpp` Vulkan runtime artifacts; broader Mesa/Venus guest-driver coverage remains future work.

Rutabaga and `gfxstream`. Rutabaga [3] is a Rust graphics virtualization abstraction layer developed for ChromeOS and Android. It unifies 2D, VirGL, Venus, and `gfxstream` paths behind a single host API, enabling `crosvm` and QEMU to share backend implementations. `Gfxstream` is a distinct protocol that serializes GLES and Vulkan calls at the API level rather than at the Gallium or `VkCommand` level, targeting low-latency Android use cases. VOGUE has no direct dependency on `rutabaga`, but `rutabaga` is the authoritative reference for how blob, `hostmem`, and `capset` semantics should behave when `virgl` is implemented.

vhost-user-gpu. The `vhost-user-gpu` mechanism [24] moves the graphics backend out of QEMU into a separate process, communicating over a UNIX socket with file-descriptor passing for `DMABUF` sharing. This isolation boundary allows the backend renderer to crash or be

replaced without affecting the VMM or guest. It is the preferred architectural boundary for production deployments and is the natural host-side target for a GGML-VirtGPU-style compute backend: the backend process can link against any host library (CUDA, ROCm, Metal, BLAS) without contaminating QEMU. VOGUE’s current evaluation uses direct QEMU `virtio-gpu-gl`; a `vhost-user-gpu` backend would be the correct isolation design for any future compute-remoting paper claim.

13.3 Guest Graphics Stacks

Linux DRM/KMS and libdrm. The Linux DRM subsystem provides the guest-side kernel driver (`virtio_gpu.ko`), GEM object management, KMS scanout, and the `libdrm` userspace API. It is the baseline guest stack for all VirGL and Venus evaluations on Linux. VOGUE explicitly replaces this stack with `libukdrm_compat` and `libukgbm_compat`, which expose the minimal API surface needed for `kmscube` and `glmark2` to compile and link without importing DRM `ioctl`s, device files, or GEM. The compatibility boundary is intentionally narrow: API compatibility, not implementation correctness for the full DRM ABI.

Mesa, GBM, EGL, and GLES. Mesa [18] is the open-source userspace OpenGL/Vulkan stack on Linux. For VirtIO-GPU, Mesa provides the VirGL Gallium driver (guest-side command encoder), the Venus driver, the GBM/EGL window system integration layer, and software rasterizers (`llvmpipe`, `softpipe`, `OSMesa`). The VirGL Gallium driver inside Mesa is the reference implementation of the guest command encoder that VOGUE must eventually reimplement in `libukvirtio_gpu`. Rather than porting Mesa — its build system, LLVM dependency, and size are incompatible with Unikraft’s link-time elimination model — VOGUE implements its own narrower encoder targeting only the Gallium commands required for the target workloads. `libukegl` and the GBM shim serve the same function as Mesa’s EGL/GBM for the software-render path, bridging application code to the VOGUE display pipeline without Mesa’s broader dependency chain.

OpenGL and Vulkan. OpenGL [9] is the principal API exposed through the VirGL path. Vulkan [10], the modern explicit API, is exposed through Venus. For VOGUE’s current executable workloads, OpenGL ES 2.0 suffices (`kmscube`, `glmark2`), while Vulkan/Venus is tracked as a readiness-gated next stage rather than a completed runtime. For compute applications (LLMs), Vulkan compute shaders and SPIR-V provide a vendor-neutral path, but they require the full Venus protocol and Mesa Vulkan guest stack — a substantially larger undertaking than the VirGL/GLES2 path.

13.4 GPU Passthrough

VFIO and vfiopci. VFIO [15] provides IOMMU-protected direct passthrough of physical PCIe devices to guest VMs. A guest with VFIO passthrough can run a full vendor GPU driver stack (CUDA, ROCm, Metal via translation) at near-native performance. However, VFIO passthrough requires

PCI enumeration, BAR mapping, MSI/MSI-X interrupt routing, and a complete vendor driver inside the guest. None of these components exist in a standard Unikraft build, and their introduction would consume tens of megabytes of code, violating unikernel minimality constraints. VFIO passthrough is the correct answer for performance-critical GPU workloads where isolation requirements permit it, but it is explicitly out of scope for VOGUE.

Confidential GPU. NVIDIA H100 and successor GPUs support a Confidential Compute mode [19] that establishes a protected channel between CPU and GPU TEE, allowing GPU computation to proceed without the host hypervisor observing GPU-resident data. This path requires hardware support, vendor-specific firmware, and a confidential VM setup incompatible with general-purpose VirtIO-GPU. Confidential GPU is the correct architecture for privacy-sensitive AI workloads (e.g., fine-tuning on regulated medical or financial data inside a CVM), but it is beyond VOGUE’s current scope. The interaction with VirtIO-GPU is also adversarial: VirtIO shared memory inherently exposes guest buffers to the host [23], weakening confidentiality guarantees that a confidential GPU is meant to provide.

13.5 LLM Compute Remoting

llama.cpp and GGML. `llama.cpp` and its underlying GGML tensor library [6] constitute the primary CPU-capable LLM inference stack in C/C++. GGML supports pluggable compute backends: CPU (reference), CUDA, Metal, Vulkan, and SYCL. Because GGML can run entirely on CPU with no GPU dependency, it is the most tractable LLM workload to run inside a Unikraft appliance today. The substrate evidence rows (`gfx.kmscube.sw`, `gfx.glmark2.sw`) validate the application framework on which an `app-ukllm` port would build.

GGML-VirtGPU. GGML-VirtGPU [7] is a `llama.cpp` backend that offloads GGML tensor operations to host GPU hardware via the VirtIO-GPU and `virglrenderer` API-remoting (Vulkan/Venus) path. The guest GGML backend packs tensor operation descriptors into `virglrenderer` API-remoting calls; the host-side renderer dispatches these to CUDA or the host GGML backend, returning results through shared memory. This design closely parallels VOGUE’s architecture for graphics: a minimal guest-side protocol library replaces the Linux DRM+Mesa guest stack. The critical difference is that GGML-VirtGPU assumes the Linux guest `virtio_gpu` DRM driver and `libdrm` `ioctl`s; replacing that dependency with `libukvirtio_gpu` and `libukdma` is the planned VOGUE extension. The DMA buffer semantics of `libukdma` directly map to the shared-memory buffer management that GGML-VirtGPU requires for zero-copy tensor sharing between guest and host.

Virglrenderer Vulkan/Venus. The API-remoting trampoline in `virglrenderer` provides the host-side dispatch mechanism for both GGML-VirtGPU and hypothetical future Unikraft compute offload. The guest issues opaque

Table 27: Implementation references for each VOGUE layer. The “Reference” column names the upstream spec or source file used; “What It Specifies” describes the specific mechanism VOGUE depends on.

VOGUE Layer	Reference	What It Specifies
<i>libukvirtio_gpu — 2D commands</i>	OASIS VirtIO Specification v1.3, §7.8 “virtio-gpu device”	Resource lifecycle commands (RESOURCE_CREATE_2D, ATTACH_BACKING, TRANSFER_TO_HOST_2D, SET_SCANOUT, RESOURCE_FLUSH), fence semantics, feature bits
<i>libukvirtio_gpu — 3D / blob</i>	VirtIO-GPU 3D extension; virtio_gpu_proto.h in Linux kernel tree include/uapi/linux/virtio_gpu.h	CTX_CREATE, SUBMIT_3D, RESOURCE_CREATE_3D, RESOURCE_CREATE_BLOB, RESOURCE_MAP_BLOB, RESOURCE_UNMAP_BLOB, UUID; feature bits VIRTIO_GPU_F_VIRGL, VIRTIO_GPU_F_RESOURCE_BLOB, VIRTIO_GPU_F_CONTEXT_INIT, VIRTIO_GPU_F_BLOB_ALIGNMENT (bit 5); virtio_gpu_config is now 20 bytes (5 fields) including blob_alignment at offset 16
<i>libukvenus — Venus ring protocol</i>	Mesa source src/virtio/vulkan/vn_ring.h and vn_ring.c; KhronosGroup/venus-protocol include/vn_protocol_driver_defines.h	Ring head/tail/status layout at offsets 0/64/128/192; circular write with wrap-around; vkCreateRingMESA / vkNotifyRingMESA / vkDestroyRingMESA entry points; packed struct layout with no inter-field padding
<i>libukvenus — Venus command encoding</i>	Mesa src/virtio/vulkan/vn_protocol_driver_*.h; vn_encode_array_size() in vn_cs.h	Array presence encoded as uint32_t n; zero = absent; all structs packed; command IDs from vn_protocol_driver_defines.h
<i>libukggml_vulkan — static Vulkan dispatch</i>	Vulkan 1.3 specification §3.2 “Command Function Pointers”; Vulkan-Headers v1.3.352 vulkan/vulkan.h	80+ Vulkan entry points; vkGetDeviceProcAddr dispatch table; VULKAN_HPP_DEFAULT_DISPATCHER initialization pattern
<i>apps/app-llama-upstream* — in-tree ggml</i>	llama.cpp docs/build.md; Unikraft APG porting guide; upstream SHA fcae601e4	In-tree source compilation pattern; Makefile.uk addlib_s + per-file SRC-C/SRC-CXX; upstream lib-musl supplies mmap/sysconf/getauxval/prctl/pthread_setaffinity_np
<i>apps/app-llama-upstream-vk — Vulkan ggml</i>	ggml-vulkan backend docs/backend/VULKAN.md; SPIR-V headers (KhronosGroup/SPIRV-Headers)	GGML_USE_VULKAN=1; uk_ggml_vk_loader.cpp wires VULKAN_HPP_DEFAULT_DISPATCHER; SPIR-V shader table requires SPIR-V headers at build time
<i>kraft/Kraftfile.llama-upstream-* — 9pfs mount</i>	Unikraft lib-9pfs documentation; CONFIG_LIBVFSCORE_AUTOMOUNT_CI_RAMFS; VirtIO-9P QEMU device (-virtfs)	CONFIG_LIBRAMFS=y, CONFIG_LIBVFSCORE_AUTOMOUNT_CI=y, CONFIG_LIBVFSCORE_AUTOMOUNT_CI_RAMFS=y required before mkdir/mount; CONFIG_LIBUK9P=y, CONFIG_LIBVIRTIO_9P=y, CONFIG_LIB9PFS=y for 9pfs transport
<i>QEMU virtio-gpu-gl-pci,venus=true</i>	QEMU 11.0 release notes; QEMU docs/system/devices/virtio-gpu.rst	-device virtio-gpu-gl-pci,hostmem=512M,blob=true,venus=true enables Venus capset id=4; -display egl-headless,gl=on (or another GL-capable backend) is required by QEMU virtio-gpu-gl before Venus initializes
<i>virglrenderer Venus decoder</i>	virglrenderer source src/venus/; vkr_context_submit_cmd()	Decodes SUBMIT_3D payload as Venus commands; expects packed layout; context initialized via DRM_IOCTL_VIRGTGPU_CONTEXT_INIT with capset id=4

context commands (SUBMIT_3D with Vulkan/Venus-encoded payloads); virglrenderer deserializes and dispatches them to registered backends. This mechanism allows a single VirtIO-GPU frontend to serve multiple use cases — graphics, compute, ML inference — purely by negotiating different capsets and command encodings. VOGUE’s libukvirtio_gpu already implements the transport-level infrastructure (CTX_CREATE, SUBMIT_3D wire format); the remaining work is encoding the Vulkan/Venus command payload correctly.

vAccel. vAccel [29] is a modular hardware acceleration framework designed to expose heterogeneous accelerators

(GPU, FPGA, NPU) to virtualized and containerized workloads through a unified API. Rather than remotng a specific GPU API, vAccel defines its own higher-level operations (image classification, object detection, generic tensor compute) and provides backend plugins for CUDA, TensorFlow, TensorRT, and others. VOGUE and vAccel occupy adjacent design points: VOGUE implements the low-level VirtIO-GPU transport for graphics and compute, while vAccel abstracts over multiple accelerator types at a higher semantic level. A future integration could route vAccel operations through the VirtIO-GPU Vulkan/Venus path, using libukvirtio_gpu as the transport.

Cricket. Cricket [13] is a CUDA virtualization layer providing transparent remote execution and checkpoint-restart for CUDA applications. It intercepts CUDA driver API calls, serializes them, and dispatches to a remote GPU over a network transport. Cricket is CUDA-specific and much heavier than VirtIO-GPU Vulkan/Venus remotng: it requires a full CUDA runtime on both ends, whereas VirtIO-GPU Vulkan/Venus operates at a transport level that is API-agnostic. Cricket is relevant primarily as a performance baseline for GPU API remotng latency and bandwidth benchmarks.

vLLM, SGLang, and TensorRT-LLM. Production LLM serving stacks such as vLLM, SGLang, and TensorRT-LLM implement advanced schedulers, continuous batching, KV-cache management, and quantized inference against CUDA or ROCm. These are host-side systems that VOGUE does not attempt to port; they serve as external performance baselines for throughput and latency under the same LLM workloads that GGML-VirtGPU targets. The relevant comparison axis is: what fraction of host GPU throughput is preserved after the VirtIO-GPU Vulkan/Venus remotng overhead? That question cannot be answered without a working GGML-VirtGPU + libukvirtio_gpu stack, which makes it K1-equivalent future work.

13.6 Summary and Position of VOGUE

13.7 Implementation References

This section records the upstream specifications, header files, and documentation used to implement each VOGUE layer. These are the authoritative sources a future implementer should consult to reproduce or extend the work.

The central dependency for K1 accelerated rendering is now satisfied on the evaluation host: xport.qemu-vgpu, Venus bootstrap command serialization, the Mesa ring-buffer protocol, and the KMSCube submit/frame rows all have current same-run evidence. The architecture VOGUE has built for the 2D path (DMA buffer management in libukdma, protocol command staging, fence synchronization in libukvirtio_gpu) is the same architecture required for 3D; future work is broader app coverage and performance rather than first transport proof.